

Dossier E5 — Decoding Speculativo su SSM

Estrazione dalla letteratura per regime CPU memory-bound (Zen 2, single-user)

Brief n.1 — da incollare al Ricercatore

Phase 62 — Design Gate (Cat-A)

Modello target: SSM stile Mamba-1 + SWA + MLP gated-dReLU ternario

Data: 2026-07-04

Sintesi fattuale dei findings

Questo dossier estrae dalla letteratura recente (preprint ultimi 18 mesi + paper peer-reviewed fondativi) i dati pubblicati su decoding speculativo e a-blocchi, con focus sulla trasferibilità al regime CPU memory-bound single-user con modello SSM selettivo. Tutti i numeri riportati sono etichettati come **[MISURATO]** o **[PROIETTATO]** e associati alla rispettiva baseline. Non contiene raccomandazioni architetturali — la decisione di design è a valle, con i dati Cat-A di Phase 62.

Punti chiave (solo fatti, no raccomandazioni)

- **Evidenza teorica bandwidth-amortization: FORTE.** Cinque fonti indipendenti formulano esplicitamente l'argomento che spec-decode ammortizza il weight-stream, non il compute. Qualcomm Cloud AI 100 blog fornisce la formula first-order $t_{ok}/s = \text{DRAM-BW} / \text{model-size}$; SpecMamba (arXiv 2509.19873) dichiara che "*Mamba autoregressive generation remains memory-bound*"; STree modella Wall Time $\propto t/\tau$ con $t = kN + C$ dove $C = \text{weight-loading cost}$.
- **Evidenza empirica DIRETTA su CPU consumer x86 single-user con SSM: ASSENTE.** Nessun paper peer-reviewed misura spec-decode su CPU consumer x86 single-thread single-user con modello SSM selettivo. L'evidenza diretta è limitata a: (a) un blog vendor OpenInfer (ago 2025) su Intel Core Ultra 7 — throughput DECREASE su CPU (compute-bound), speedup sulla iGPU (memory-bound); (b) Ghidorah (arXiv 2505.23219) su edge ARM CPU+GPU single-sample, $7.6\times$ speedup (di cui $3.27\times$ algoritmico puro).
- **Compatibilità SSM: 4 paper con misurazioni dirette.** Mamba Drafters (drafter Mamba-130M + verificatore Transformer, GPU H100), Snakes and Ladders (drafter n-gram + verificatore Mamba, GPU batched), STree (drafter Mamba2-130M distillato + verificatore Mamba2InLlama-8B, GPU 3090 single-user), Mamba in the Llama (drafter Transformer 2/4-layer + verificatore Mamba2 hybrid, GPU 3090 single-user). **Nessun paper testa Mamba-drafter + Mamba-verifier con pesi condivisi** (self-speculative Mamba puro) — esplicitamente future work in Mamba Drafters.
- **Gestione stato su accettazione parziale (SSM-specifico): due approcci documentati.** (a) **Activation Replay** (Snakes and Ladders, STree): cache di attivazioni intermedie del blocco + replay per ricomputare lo stato fino all'ultimo accettato. (b) **Snapshot h_j + MultiStep kernel** (Mamba in the Llama): snapshot dello stato SSM pre-blocco + kernel fused recompute/decode/cache. In entrambi i casi il ricalcolo è inevitabile per SSM (a differenza di Transformer dove basta KV-cache slicing), ma reso near-free via kernel fusion su GPU.
- **Ceiling algoritmico puro convergente a 2–3.3×.** Ghidorah $3.27\times$ (MBPP, width=16); SpecDec-Quant $2.78\times$ (Llama-3-70B W4A16, A100); STree $1.36\text{--}1.98\times$ (MambaInLlama-8B); SpecMamba $2.27\times$ (FPGA); Staged Spec Dec $3.16\times$ (GPT-2-L); Qualcomm SpD $\sim 2\times$ (Cloud AI 100, batch=1). Il soffitto onesto pre-registrato $\approx 2\text{--}4\times$ del brief è **allineato o leggermente conservativo** rispetto all'evidenza pubblicata.

- **Self-speculative su fondazione piccola (~5M): GAP TOTALE.** Nessuno dei 6 paper self-speculative (LayerSkip, CALM, SWIFT, Medusa, EAGLE, QuantSpec) testa su modelli <1B parametri. Taglia minima testata: Llama-3.2-1B (LayerSkip, early-exit @ layer 4, 1.80× efficiency). SWIFT è l'unico zero-shot plug-and-play (1.3–1.6×), ma non testato <1B né su CPU né su SSM.

1. Domanda 1 — Paper DeepSeek su speculative decoding

Il brief cita un claim DeepSeek di speedup 51%–400% per speculative decoding. La ricerca ha identificato due paper rilevanti: **DeepSeek-V3 Technical Report** (dic 2024), che introduce i moduli MTP (Multi-Token Prediction) come meccanismo di self-speculative decoding co-trained con il modello target, e **DSpark** (giu 2026), che introduce un drafter semi-autoregressive attachable con confidence-scheduled verification per DeepSeek-V4. DeepSeek-R1 (gen 2025) è stato verificato come **non rilevante** — è un paper puramente RL, nessun claim spec-decode.

1.1 DeepSeek-V3 — Multi-Token Prediction (MTP)

Citazione: [1] DeepSeek-AI (Liu, A. et al.), *DeepSeek-V3 Technical Report*, arXiv:2412.19437, dicembre 2024. URL: <https://arxiv.org/abs/2412.19437> — codice: <https://github.com/deepseek-ai/DeepSeek-V3>.

Metodo esatto. I moduli MTP non sono un drafter separato: ogni MTP module è un Transformer block aggiuntivo che **condivide embedding e output head** con il modello principale. Profondità MTP in produzione = 1 (predict next 2 tokens). Implementazione ispirata a EAGLE ma con obiettivo primario dichiarato di *densificazione del training signal*, non spec-decode. La causal chain è mantenuta esplicitamente per ogni depth (come EAGLE). Verificatore = modello target stesso (DeepSeek-V3): approccio **self-speculative via MTP head**.

Regime del claim. Hardware: GPU NVIDIA H800 (cluster 2048 GPU per training). Inference deploy su multi-GPU, dimensione non specificata. Batch: non riportato esplicitamente (deployment production-scale batched serving, **non single-user**). Modello target: DeepSeek-V3, MoE 671B totali / 37B attivati per token, architettura MLA + DeepSeekMoE. **Baseline esplicita:** vanilla autoregressive decoding dello stesso modello senza testa MTP.

Lunghezze di accettazione. "The acceptance rate of the second token prediction ranges between 85% and 90% across various generation topics" — **[MISURATO]**, baseline = vanilla autoregressive, dominio = various generation topics (NON disaggregato per codice/prosa/reasoning/math). Speedup: **1.8× TPS (Tokens Per Second)** — **[MISURATO]**, baseline = vanilla autoregressive. Acceptance length esplicita $a = k$ non riportata in forma tabellare; implicitamente $a \approx 1 + 0.85 \div 0.90 = 1.85 \div 1.90$ (2 token proposti, ~85-90% di hit sul secondo).

Costo retrofit vs co-training. Co-training **obbligatorio**: i moduli MTP sono addestrati congiuntamente al main model durante il pretraining (loss L_{MTP}^k con peso $\lambda = 0.3$ per i primi 10T token, poi 0.375). **Retrofit NON testato** nel paper. Il paper dice solo che i moduli MTP *possono*

essere scartati e il modello funziona normalmente, OPPURE riproposti per spec-decode. Overhead di inference: ogni MTP module = 1 Transformer block aggiuntivo + testa condivisa; memoria overhead non quantificata esplicitamente. **Limite dichiarato:** *"recommended deployment unit for DeepSeek-V3 is relatively large, which might pose a burden for small-sized teams".*

Rilevanza single-user CPU: PARZIALE (tendente a NO). Il paper misura su GPU H800 datacenter con modello 671B MoE e deployment multi-GPU; il claim 1.8× è in regime batched serving, non single-user CPU. Inoltre l'MTP è co-trained da zero → inapplicabile come retrofit a modelli SSM CPU esistenti.

Gap di evidenza. Nessuna misura CPU o single-user; nessuna misura su modelli dense non-MoE; nessuna disaggregazione per dominio (codice vs prosa vs reasoning); nessuna acceptance length esplicita per benchmark type (HumanEval, GSM8K, etc.); nessun dato su modelli <10B (regime tipico CPU single-user); retrofit di MTP head su checkpoint esistente non testato; nessuna misura in regime memory-bound esplicito.

1.2 DSpark — Confidence-Scheduled Speculative Decoding

Citazione: [2] Cheng, X. et al. (Peking University; DeepSeek-AI), *DSpark: Confidence-Scheduled Speculative Decoding with Semi-Autoregressive Generation*, 27 giugno 2026. URL: <https://www.alphaxiv.org/abs/2026.dspark> — codice: <https://github.com/deepseek-ai/DeepSpec> (MIT license).

Metodo esatto. Drafter separato ("DSpark head") **attachable**, non DeepSeek-specifico: con i draft checkpoints giusti funziona anche su Qwen e Gemma. Architettura **semi-autoregressive**: combina (i) un parallel backbone (emette tutti i token di un blocco in un singolo forward pass, come DFlash) + (ii) un lightweight sequential/serial head che introduce dipendenza intra-block per mitigare il *suffix decay* (problema dei parallel drafter in cui i token di coda del blocco vengono rigettati perché generati senza guardare i token precedenti del blocco stesso). Verificatore = modello target = DeepSeek-V4 (Pro 1.6T totali / 49B attivati; Flash 284B totali / 13B attivati). MoE. Verifica **confidence-scheduled**: una confidence head score ogni token proposto; uno scheduler hardware-aware decide la lunghezza del blocco da verificare in base al carico del server. **Lossless**: output byte-for-byte identico al target.

Regime del claim. Hardware: GPU datacenter (deployment su DeepSeek-V4 serving system under live user traffic; NVIDIA forum conferma DGX Spark / GB10). Batch: **batched serving under live production traffic** (alta concorrenza, NON single-user). Il paper mette esplicitamente in foreground la gestione del batch capacity (*"wastes critical batch capacity on tokens with high rejection risks"*). Context length 1M token. **Baseline esplicita:** MTP-1 (production baseline di DeepSeek-V4 = single-token MTP, equivalente al setup di DeepSeek-V3). Confronto anche vs EAGLE-3 e DFlash su Qwen-3.

Lunghezze di accettazione. Claim paper: *"DSpark accelerates per-user generation speeds by 60%–85% at matched throughput levels"* vs MTP-1 baseline — **[MISURATO]**, baseline = MTP-1, dominio = produzione live traffic (non disaggregato per codice/prosa/math). Range "51%–400%": **NON appare nell'abstract** del paper. Fonti terze (blog explainx.ai, deepseek.ai fan blog) lo attribuiscono a DSpark: la fonte blog chiarisce *"The wider 50 to 400% range you'll sometimes see*

refers to corner cases on the serving frontier; 57–85% is the typical, conservative outcome". Etichetta: **[MISURATO su corner-case] / [PROIETTATO per il range esterno]**. Accepted length relativo: "DSpark accepts roughly 30% longer blocks than Eagle 3 and 16–18% more than DFlash" su Qwen-3 — **[MISURATO]**, baseline = Eagle 3 / DFlash. Numeri esatti di $a = k$ per benchmark (HumanEval, GSM8K, etc.) non visibili nell'abstract; tabella dettagliata descritta in blog di terzi ma da verificare nel full-text del paper.

Costo retrofit vs co-training. Retrofit **SÌ**, **esplicitamente supportato**. Dal paper: "DSpark head is a single attachable component rather than a DeepSeek-specific architecture. With the right draft checkpoints it also works on Qwen and Gemma". Il drafter è un modulo addestrabile separatamente e attaccabile a target model esistenti. Co-training **NON** richiesto. Overhead di inference: il draft head deve essere ~10-30× più veloce del target per rendere (fonte: FAQ blog di terzi, non paper). Memoria overhead = dimensione del draft head + serial head, non quantificata esplicitamente nell'abstract. **Limiti dichiarati:** *acceptance rates collapse on out-of-distribution prompts*; range di throughput molto ampio (51-400%) perché altamente workload-dependent.

Rilevanza single-user CPU: NO. Il paper misura esplicitamente in regime batched serving under live traffic su GPU datacenter con MoE 1.6T/49B o 284B/13B. Il contributo centrale (confidence-scheduled verification) è specificamente pensato per *high-concurrency serving systems* — il problema dell'allocazione del batch capacity sotto carico è inesistente in regime single-user.

Gap di evidenza. Nessuna misura CPU o single-user; nessuna misura su modelli dense non-MoE; nessuna misura in regime memory-bound esplicito (il paper parla di *low GPU utilization* ma in regime compute-underuse, non memory-bandwidth-bound CPU); tabella di acceptance length per benchmark (codice/prosa/reasoning/math) non visibile nell'abstract; nessuna misura su modelli SSM o architetture ibride (il target è Transformer MoE); costo di training del draft head (GPU-ore, dataset) non riportato; overhead di memoria del draft head non quantificato esplicitamente; il range 51-400% non è direttamente attribuibile al paper — è una rielaborazione di blog di terzi da verificare nel full-text.

1.3 DeepSeek-R1 — escluso

Citazione: DeepSeek-AI, *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*, arXiv:2501.12948, 22 gennaio 2025. URL: <https://arxiv.org/abs/2501.12948>. L'abstract e i metadati **non contengono alcuna menzione** di *speculative decoding*, *MTP*, *multi-token prediction*, *drafter*, *speedup*, $1.8\times$ o $1.5\times$. DeepSeek-R1 è un paper puramente RL (reinforcement learning per incentivizzare il reasoning, con distillazione in modelli più piccoli). **Non rilevante per il dossier.**

2. Domanda 2 — Spec/block decoding su SSM/Mamba

Quattro paper rilevanti identificati. Coprono: (a) drafter SSM con verificatore Transformer (Mamba Drafters), (b) drafter n-gram con verificatore Mamba (Snakes and Ladders), (c) drafter SSM distillato + verificatore Mamba2 ibrido (STree), (d) drafter Transformer + verificatore Mamba2 ibrido

(Mamba in the Llama). **Nessun paper Jamba-specifico su spec-decode trovato** (Jamba citato solo come architettura ibrida in letteratura). **Nessun paper in regime CPU single-user** — vedi Flag gap sotto.

2.1 Mamba Drafters for Speculative Decoding

Citazione: [3] Choi, D.; Oh, S.; Dingliwal, S.; Tack, J.; Kim, K.; Song, W.; Kim, S.; Han, I.; Shin, J.; Galstyan, A.; Katiyar, S.; Bodapati, S.B. (KAIST; Amazon AGI; Seoul National University), *Mamba Drafters for Speculative Decoding*, arXiv:2506.01206v1, giugno 2025 (EMNLP 2025 Findings). URL: <https://arxiv.org/abs/2506.01206>.

Metodo esatto. Drafter = Mamba-130M **addestrato from scratch** su FineWeb-Edu (5.000 step, batch 4.096, ctx 2k) + SFT su ShareGPT per 2 epoche. **NON è distillazione dal target.** Drafter target-agnostic (cross-target). Verificatore = Transformer (Mistral-7B-Instruct, Pythia-6.9B) — **NON SSM, NON Mamba-2.** Spec-decode classico con **tree-structured drafting via batch generation**: sfrutta il fatto che Mamba ha stato fisso (copia stato corrente e sample, niente KV-cache da duplicare). Aggiunge **test-time tree search via multi-armed bandit (UCB)** che sceglie dinamicamente la configurazione dell'albero.

Gestione stato su accettazione parziale (SSM). Non affrontato esplicitamente per SSM come verificatore: poiché il verificatore è Transformer (stato = KV-cache, rollback banale tramite slicing), il problema del rollback stato SSM non si pone. Lo stato del **drafter** Mamba viene copiato per branching (batch generation), ma non è documentato alcun meccanismo di rollback esplicito dello stato SSM in caso di accettazione parziale — non necessario perché il drafter SSM rigenera lo stato dal prefisso accettato.

Accettazioni misurate [MISURATO], baseline = no-drafter / EAGLE / external Transformer drafter. HumanEval (codice), target Pythia-6.9B, Mamba-130M drafter (temp=1): throughput **128.21 tok/s**, accept length **3.91** (vs EAGLE 125.61 / 3.85; vs no-drafter 54.21). HumanEval, target Mistral-7B, Mamba-130M: **118.04 tok/s, 3.16** (vs EAGLE 107.16 / 3.22; vs no-drafter 52.34). GSM-8K (math), target Pythia-6.9B: **149.46 tok/s, 4.57** (vs Pythia-410M external 75.97 / 4.64) — throughput ~2× superiore. XSum/CNN-DM (prosa), target Pythia-6.9B greedy: **138.80/131.97 tok/s, 4.55/4.38**. Long-context (LongBench single-doc QA, target Mistral-7B), 1k → 8k: Mamba accept length **2.91 → 2.80** (stabile), throughput 55.09 → 24.92 tok/s; EAGLE 2.73 → 2.71. Memoria: a ctx=8k su LongBench multi-doc QA, target+draft peak = **52 GB** per Mamba vs **72 GB** per EAGLE (−20 GB).

Regime del claim. Hardware: NVIDIA H100. Batch: **single-user inference** (tree = batch interno per i branch del drafter, non multi-utente). Dimensione modello target: Mistral-7B, Pythia-6.9B; drafter 130M Mamba.

Costo drafter/retrofit. Drafter addestrato from scratch su FineWeb-Edu + SFT ShareGPT, separato dal target. **Retrofit sì in senso lato** — drafter target-agnostic, riutilizzabile su target diversi senza riaddestramento (vs EAGLE che va riaddestrato per ogni target). Overhead inference: memoria (peak −20 GB vs EAGLE a ctx=8k); latenza drafting Mamba-130M comparabile a EAGLE e nettamente inferiore a Mistral-160M external.

Rilevanza single-user CPU: NO. Tutti i numeri sono GPU (H100). Il claim di efficienza del drafter SSM (memoria costante, latenza bassa) è qualitativamente favorevole a regime memory-bound, ma nessuna misura CPU. Inoltre il **verificatore è Transformer**, quindi il caso d'uso CPU memory-bound single-user su puro SSM non è coperto. Self-speculative Mamba (drafter e verificatore entrambi SSM, condividendo pesi) è esplicitamente elencato come **future work** dagli autori.

Gap di evidenza. Nessuna misura su CPU; verificatore è Transformer, non SSM (il caso *SSM come verificatore a-blocchi* non è testato in questo paper); nessuna analisi di regime memory-bound DRAM (tutto HBM/SRAM); long-context testato solo fino a 8k token; self-speculative Mamba (drafter e verificatore entrambi SSM con pesi condivisi) è future work dichiarato.

2.2 Snakes and Ladders — Accelerating SSM Inference

Citazione: [4] Wu, Y.; Dukler, Y.; Trager, M.; Achille, A.; Xia, W.; Soatto, S. (Amazon AWS), *Snakes and Ladders: Accelerating SSM Inference with Speculative Decoding*, PMLR vol. 262, pp. 292–304, NeurIPS 4th Efficient NLP and Speech Processing Workshop, 14 dicembre 2024. URL: <https://proceedings.mlr.press/v262/wu24a.html>.

Metodo esatto. Drafter = **n-gram model** (nessuna rete neurale). Verificatore = SSM (Mamba) — case *Mamba-1 come verificatore*. Spec-decode classico con due varianti algoritmiche per SSM: (i) **Joint Attainment and Advancement** — specifica e verifica simultaneamente sfruttando compute idle; (ii) **Activation Replay** — caching di attivazioni intermedie per ricomputare lo stato. Entrambi utilizzano risorse di calcolo in idle.

Gestione stato su accettazione parziale — CRITICO. Ricalcolo del blocco esplicito via Activation Replay. Per SSM (stato Markov ht), il *rollback non è banale come per attention (KV-cache slice)*. Il metodo cache le ***attivazioni intermedie*** durante la verifica forward del blocco di draft tokens e le ***replay*** per ricomputare lo stato $h\{t\}$ corretto fino all'ultimo token accettato, senza materializzare gli stati intermedi. Checkpoint periodico non esplicitato, ma l'Activation Replay funziona da checkpoint implicito delle attivazioni del blocco in verifica. Meccanismo tecnico esatto: kernel hardware-aware che evita di scrivere lo stato SSM completo in HBM a ogni step (reduce memory transfer cost).

Accettazioni misurate [MISURATO], baseline = inference SSM senza spec-decode. Claim centrale: produrre 6 token al costo di **1.47× il costo di un singolo token** (costo relativo). Wall-clock speedup: media **1.82×** su tre benchmark (specifici non dati in abstract). Scaling target: 1.3B → 13B parametri → overhead relativo di speculation+verification scende da **1.98× a 1.22×**. Dominio: non riportato (abstract cita *three benchmarks* ma non specifica codice/prosa/math/reasoning).

Regime del claim. Hardware: GPU (riferimento a HBM, compute idle; non specificato modello). Batch: **batched**. Affermazione chiave: *"Unlike Transformers, speculative decoding in SSMs can be easily applied to batches of sequences, allowing dynamic allocation of resources to fill gaps in compute utilization and thereby improving efficiency and throughput with variable inference traffic."* Dimensione modello target: 1.3B e 13B (Mamba family).

Costo drafter/retrofit. Drafter = n-gram, costo di training nullo; nessun retrofit. Overhead inference: $1.47\times$ il costo di un singolo token per 6 token prodotti ($\approx 4\times$ più efficiente per token in termini di compute, ma include costo di replay/caching).

Rilevanza single-user CPU: PARZIALE. Il meccanismo Activation Replay (cache attivazioni + ricomputa stato) è concettualmente trasferibile a regime memory-bound, ma il paper è ottimizzato per GPU (HBM, kernel fusion). L'argomento *batched per fillare compute idle* è ortogonale al caso single-user. Nessuna misura CPU.

Gap di evidenza. Nessuna misura CPU; nessuna misura single-user (tutta l'enfasi è su batched per throughput variabile); nessun breakdown per dominio (codice/prosa/math); drafter n-gram: non rappresenta il caso d'uso con drafter neurale piccolo (Mamba-130M etc.); nessuna misura esplicita di overhead di memory transfer in regime DRAM-bound.

2.3 STree — Speculative Tree Decoding for Hybrid State-Space Models

Citazione: [5] Wu, Y.; Qin, Z.; Wong, A.; Soatto, S. (UCLA; Yale University), *STree: Speculative Tree Decoding for Hybrid State-Space Models*, arXiv:2505.14969v2, NeurIPS 2025 poster. URL: <https://arxiv.org/abs/2505.14969> — codice: <https://github.com/wyc1997/stree>.

Metodo esatto. Drafter = Mamba2-130M **distillato 2-layer dal target** (MambaInLlama-8B) usando dati di fine-tuning del target. Checkpoint a 48.000 step usato come default. Architettura target **ibrida**: Mamba2InLlama-8B (50% Mamba2 + 50% Transformer). Verificatore = **Mamba2-2.7B** (puro SSM, SSD) e **Mamba2InLlama-8B** (50% Transformer, 50% Mamba2) — case *Mamba-2/SSD come verificatore a blocchi*. Estensione **tree-based** dello spec-decode per SSM. Innovazione: **TreeScan kernel** che processa un albero di token *packed* come una singola sequenza con mask ad albero, sfruttando la struttura diagonale di A (transition matrix) di Mamba2. Metodo alternativo all'unrolling (che richiede 1 stato SSM per sequenza srotolata → esplosione di memoria).

Gestione stato su accettazione parziale — CRITICO. Activation Replay (riutilizza metodo di Snakes and Ladders): durante la verifica, il TreeScan kernel **non genera lo stato finale** (perché lo stato finale è *most likely incorrect unless all tokens accepted*). Al prossimo step, $\text{ActivationReplay}(L^*, \text{Cache})$ ricomputa lo stato corretto fino all'ultimo token accettato usando la mask L^* e la cache di attivazioni ($A\{i:j\}$, $B\{i:j\}$, $C_{-}\{i:j\}$). Rollback esplicito SSM sì, via activation replay su maschera last-accepted. Cache di attivazioni per blocco, replay on-demand. Meccanismo tecnico esatto: kernel TreeScan ritorna (t', Cache) ; algoritmo Alg.1 line 7: $x^* \leftarrow \text{ActivationReplay}(L^*, \text{Cache})$ prima di ogni nuovo drafting step. Hardware-aware: evita materializzazione stati intermedi in shared GPU memory.

Accettazioni misurate [MISURATO], baseline = Vanilla SD (1 seq × 5 token con Fused Selective Scan) / Autoregressive / FSS su unrolled tree / Chunk Scan. Target: MambaInLlama-8B (50% mix), drafter Mamba2-130M distillato, generazione 1024 token. MT-Bench (prosa/dialogo) temp=0: STree **69.84 tok/s (1.74×)**, $\tau=2.47$; Vanilla SD 67.68 (1.69×) $\tau=2.04$; AR 39.98. HumanEval (codice) temp=0: STree **78.35 (1.95×)** $\tau=2.79$; Vanilla SD 77.18 (1.93×) $\tau=2.32$; AR 39.98. GSM-8K (math/reasoning) temp=0: STree **80.03 (1.98×)** $\tau=2.83$; Vanilla SD 78.38 (1.93×) $\tau=2.34$; AR 40.47. MT-Bench temp=1: STree 54.08 (1.36×) $\tau=2.03$; Vanilla SD 49.24 (1.23×) $\tau=1.55$. HumanEval

temp=1: STree 60.34 (1.50×) τ =2.26; Vanilla SD 52.88 (1.32×) τ =1.65. GSM-8K temp=1: STree 58.25 (1.45×) τ =2.16; Vanilla SD 51.80 (1.28×) τ =1.61. Forward pass scaling (Mamba2-2.7B, full binary tree 4 → 6 layer, 15 → 63 token): STree 27.6 → 34.0 ms; FSS 27.3 → 59.8 ms. Generation speed vs unrolled FSS (Mamba2-2.7B target, M=2..5 beams, N=4..16 steps): STree **1.05× → 1.49× speedup**; memoria 0.91× → 0.57× (più basso = meglio). FSS va OOM a M=4/5, N=16 su 24 GB. Scaling dimensione modello (forward pass, Mamba2 random init): overhead STree vs AR scende **2.04× (2.3B) → 1.47× (7B) → 1.37× (13B) → 1.26× (23B)**.

Regime del claim. Hardware: Nvidia RTX 3090 (24 GB); risultati H100 in appendix. Batch: **single-user** (generazione 1024 token per singola sequenza su benchmark). Dimensione modello target: Mamba2-2.7B, MambaInLlama-8B (50% att). Forward-pass test anche su 7B/13B/23B random-init.

Costo drafter/retrofit. Drafter non co-trained; **retrofit**: 2-layer SSM distillato dal target usando dati di fine-tuning del target. Checkpoint 48.000 step. Overhead inference: STree forward pass ~1.26–2.04× più lento di AR a parità di input; ma la riduzione di wall-time deriva da τ (acceptance length). Memoria STree quasi costante al crescere dell'albero (8 → 8.3 GB per M=2..5, N=16); FSS esplode (8.5 → 14 GB, poi OOM). Drafter è piccolo (130M) → drafting latency trascurabile.

Rilevanza single-user CPU: PARZIALE. Single-user sì (generazione 1024 token). Hardware GPU (3090, H100 in appendix). Il metodo TreeScan è hardware-aware per shared-memory GPU; non testato su CPU. La memoria costante di STree cresce poco con tree size — potenzialmente favorevole in regime DRAM-bound, ma non misurato.

Gap di evidenza. Nessuna misura CPU; risultati H100 relegati in appendix; nessun test su target puro Mamba2 a 7B+ pretrained (usa random init per misurare scaling); drafter distillato *early checkpoint* (48.000 step) — accept length probabilmente migliorabile, ma non misurato il ceiling; nessun breakdown codice vs math vs prosa oltre i 3 benchmark standard; non misura Jamba o altri ibridi reali (solo Mamba2InLlama distillato da Wang et al. 2024).

2.4 The Mamba in the Llama — Distilling and Accelerating Hybrid Models

Citazione: [6] Wang, J.; Paliotta, D.; May, A.; Rush, A.M.; Dao, T. (Cornell; University of Geneva; Together AI; Princeton), *The Mamba in the Llama: Distilling and Accelerating Hybrid Models*, arXiv:2408.15237, NeurIPS 2024. URL: <https://arxiv.org/abs/2408.15237>.

Metodo esatto. Drafter = Transformer *shrink and fine-tune* (2-layer e 4-layer), inizializzato da layer di Zephyr-7B (indici [0,31] per 2-layer, [0,10,20,31] per 4-layer), embeddings e LM head da Zephyr-7B. Training 3 epoche su OpenHermes2.5 con loss masking sul prompt. Target = **hybrid Mamba/Mamba2 con 50%/25%/12.5%/0% attention layer** distillato da Llama-3-8B-Instruct o Zephyr-7B. Verificatore = **Mamba2-SSD e Mamba2-Llama3 hybrid** (case *Mamba-2/SSD come verificatore a-blocchi*). Spec-decode classico con **Multi-Step Linear RNN Speculation algorithm** (Alg. 2). Kernel hardware-aware `MultiStep(h_i, y_{1:n}, i, j, k; A, B, C, Δ)` che computa in una singola passata `y_{j:k}`, `h_j`, `h_k` senza materializzare `h_{1:n}` né i parametri discretizzati. **Assunzione esplicita del paper:** "*decoding is bottlenecked by memory and not by compute*".

Gestione stato su accettazione parziale — CRITICO. Cache singolo stato SSM + MultiStep kernel con snapshot. L'algoritmo mantiene un singolo h_cache (stato SSM cached). Alla verifica, il kernel MultiStep ritorna sia h_j (snapshot PRIMA di valutare i draft) sia h_k (stato finale dopo draft). Se tutti accettati $\rightarrow$ usa h_k . *Se conflitto a posizione $k' < k \rightarrow$ usa h_j* (snapshot allo stato prima del blocco draft). Il kernel fonde in una singola chiamata: (i) ricomputation di step precedenti dalla cache, (ii) multi-step decoding del nuovo blocco draft, (iii) caching dei nuovi step. Rollback esplicito SSM sì, snapshot h_j pre-blocco + ricalcolo. *Snapshot ad ogni iterazione (h_j cached).* Meccanismo tecnico esatto: fuso in un unico kernel con circular buffer per la parte convoluzionale del blocco Mamba (mantiene vecchie entries per ricomputazione). Drafter (Transformer) ha kernel separato fused recompute+decode+cache. **"Dissolve del rollback" su SSM:** il paper nota esplicitamente che *"even though more tokens are generated we cannot advance the hidden state cache"* dopo una rejection — il ricalcolo è inevitabile per SSM, ma reso *near-free* via kernel fusion.

Accettazioni misurate [MISURATO], baseline = target senza spec-decode. Setup: drafter Transformer 2/4-layer, K=4 draft tokens, target Mamba-Zephyr / Mamba-Llama3 hybrid, dati OpenHermes2.5, single NVIDIA RTX 3090. Mamba-Zephyr (50% att), 2-layer drafter, K=4: #gen 2.48, **speedup 1.80×**. Mamba-Zephyr (25% att), 2-layer, K=4: #gen 2.64, **1.88×**. Mamba-Zephyr (50% att), 4-layer, K=4: #gen 3.00, **1.81×**. Mamba-Zephyr (25% att), 4-layer, K=4: #gen 3.00, **1.80×**. Mamba-Llama3 (50% att), 4-layer, K=3: #gen 2.70, **1.60×**. Mamba-Llama3 (50% att), 4-layer, K=4: #gen 3.60, **1.58×**. Throughput assoluto Mamba 7B: >300 token/sec (claim generale, non specificato il setup). Naive impl su 3090: 1.5× speedup (Mamba 2.8B, 60% acceptance). **Naive impl su H100: nessuno speedup** (GEMM troppo veloce, overhead caching/recompute domina) $\rightarrow$ kernel fusion necessaria. Dominio: non riportato (tutto su OpenHermes2.5 chat continuations).

Regime del claim. Hardware: NVIDIA RTX 3090 (spec-decoding experiments), H100 (speedup analysis), A100 8×80 GB per training (distillation). Batch: **single-user** (generazione chat continuations). Dimensione modello target: 7B (Mamba-Zephyr, Mamba-Zephyr hybrid), 8B (Mamba-Llama3 hybrid). Drafter 2/4-layer piccolo (~100M parameter range, non esplicitato).

Costo drafter/retrofit. Drafter non co-trained; **retrofit:** drafter Transformer 2/4-layer addestrato dopo la distillazione del target, su OpenHermes2.5 per 3 epoche, con init da layer di Zephyr-7B. Overhead inference: su H100 il naive impl è **peggio del baseline** (nessuno speedup) — richiede kernel fusion su misura. Su 3090, naive impl già da 1.5×. Costo di training del drafter: 3 epoche su OpenHermes2.5 (modesto). Costo distillazione target: <5 giorni su 8×A100 80GB per 20B token.

Rilevanza single-user CPU: PARZIALE. Single-user sì (chat continuations). Hardware GPU (3090/H100). L'assunzione *memory-bound non compute-bound* è qualitativamente trasferibile a CPU DRAM-bound, ma nessuna misura CPU. Il fallimento su H100 (regime meno memory-bound) suggerisce che il vantaggio è massimo dove memory-bound è più severo — CPU potrebbe essere il caso migliore, ma non testato.

Gap di evidenza. Nessuna misura CPU nonostante l'assunzione memory-bound sia centrale; H100 con naive impl non dà speedup $\rightarrow$ richiede kernel fusion, il vantaggio *naturale* del multi-step su SSM in regime memory-bound è dimostrato solo indirettamente (3090 vs H100); nessun breakdown per dominio (codice vs prosa vs math vs reasoning); solo K=3 e K=4 (draft length) testati; nessuna analisi

di scaling del draft length; drafter è Transformer, non SSM (simmetria mancante: perché non Mamba drafter + Mamba verifier?); solo modelli 7–8B testati; nessun dato scaling.

3. Domanda 3 — Trasferibilità al regime CPU/memory-bound

Il brief chiede se esista evidenza pubblicata di guadagni da decoding speculativo/a-blocchi dove il vantaggio è l'ammortamento dello stream dei pesi (non i FLOP di batch su GPU). La ricerca ha identificato **4 fonti primarie** (Ghidorah, OpenInfer blog, SpecDec-Quant, Qualcomm Cloud AI 100 blog) + numerose fonti di supporto indiretto. **Conclusione anticipata:** l'argomento teorico bandwidth-amortization è **FORTE e convergente** (≥ 5 fonti indipendenti con formule quantitative coerenti), ma l'evidenza empirica **diretta** su CPU consumer x86 single-user con modello SSM è **ASSENTE**. Il soffitto numerico $\approx 2\text{--}4\times$ del brief è allineato all'evidenza pubblicata.

3.1 Ghidorah — Edge LLM Inference with Speculative Decoding

Citazione: [7] Wei, J.; Huang, Y.; Zhou, Y.; Jiang, J.; Du, J., *Ghidorah: Fast LLM Inference on Edge with Speculative Decoding and Hetero-Core Parallelism*, arXiv:2505.23219v1, 29 maggio 2025. URL: <https://arxiv.org/abs/2505.23219>.

Regime sperimentale. Hardware: NVIDIA Jetson Xavier NX (CPU-GPU edge device, unified memory). 384-core Volta GPU locked a 204 MHz + 6-core ARM v8.2 CPU locked a 1.9 GHz (volutamente degradato per simulare end-user device bilanciato). Unified memory architecture (CPU e GPU condividono DRAM, no off-chip transfer). Memory bandwidth: non riportata esplicitamente in GB/s; architettura unified-memory con sync pagina < 0.1 ms tra unità. Il paper dichiara esplicitamente: *"The decoding phase is heavily constrained by the memory bandwidth and cannot fully exploit hardware capability"* e *"the sequential decoding approach is memory-bandwidth-bound"*. Batch size: **single-sample inference esplicita (batch=1):** *"end-user devices typically process only a single request at a moment"; "single-sample inference processes only a single token during each iteration of the decoding phase"*. Modello target: Vicuna-7B (fine-tune di LLaMA-7B) con Medusa-5head.

Misurazioni [MISURATO], baseline = Sequential (GPU-only) + Medusa + Medusa+EM. Speedup dichiarato: fino a **7.6 $\times$** sulla fase di decode rispetto al sequential decoding su Jetson NX, a verification width = 16. Decomposto dall'autore in **3.27 $\times$ algorithmic enhancement (acceptance length) $\times$ 2.31 $\times$ parallel speedup (hetero-core)**. vs Medusa-su-GPU-only: +2.06 $\times$ (MBPP); vs Medusa+EM (existing system): +1.20 $\times$. Acceptance length misurata (Medusa 5-head, MT-Bench calibration): MT-bench — width 1/2/4/8/16/32/64 $\rightarrow$ 1.00 / 1.72 / 2.28 / 2.59 / 2.93 / 3.19 / 3.34. GSM8K — 1.00 / 1.76 / 2.43 / 2.69 / 3.08 / 3.34 / 3.56. MBPP — 1.00 / 1.78 / 2.54 / 2.89 / **3.27** / 3.55 / 3.74. HumanEval — 1.00 / 1.77 / 2.49 / 2.80 / 3.19 / 3.48 / 3.71. Memoria occupata dal drafter: non isolata (Medusa usa head aggiuntivi sullo stesso modello target, nessun drafter esterno separato).

Argomento teorico. Meccanismo: *"Since the sequential decoding approach is memory-bandwidth-bound, its execution time is similar to that of Medusa, and the acceptance length is fixed at 1. However, the Medusa method generates multiple tokens in a single step and requires the similar*

execution time." — cioè la forward pass di Medusa con verification width W costa quasi quanto la forward pass single-token perché entrambe sono bandwidth-bound sul weight loading; ergo l'acceptance length > 1 è guadagno netto. Osservazione chiave dell'autore: GPU mantiene execution time simile da width 4 a 64; CPU mantiene execution time simile solo da 4 a 16 (poi cresce) → sweet spot CPU = 16, sweet spot GPU = 64. L'autore esplicita: *"Longer acceptance lengths do not always result in better throughput... essential for the ARCA approach to balance the algorithmic parallelism with hardware capabilities"*.

Rilevanza single-user CPU: PARZIALE. Il paper è single-user (batch=1, single-sample) e memory-bandwidth-bound esplicito. MA: (i) hardware è edge SoC ARM v8.2 + Volta GPU, non CPU consumer x86; (ii) il speedup 7.6× include la 2.31× parallel speedup dal collaborative CPU+GPU che non si applica al nostro scenario single-thread puro; (iii) modello target è Transformer (Vicuna-7B), non SSM. L'algorithmic-enhancement puro (3.27×) è però direttamente comparabile al soffitto a_≈2–4× del brief.

3.2 OpenInfer Blog — Consumer CPU x86 test

Citazione: [8] OpenInfer, *Boosting Local Inference with Speculative Decoding*, Engineering Blog, 5 agosto 2025. URL: <https://openinfer.io/news/2025-08-05-boosting-local-inference-with-speculative-decoding>. Fonte vendor blog, **non peer-reviewed**, ma unica fonte primaria di misure su consumer hardware x86.

Regime sperimentale. Hardware: consumer-grade. *"Intel Core Ultra 7 CPU"* + sua iGPU integrata + altre compute units. Memory bandwidth: dichiarato 128 GB/s di riferimento (esempio esplicito: *"On a system with 128GB/s memory bandwidth"*). Batch size: single-model workflows (single-user locale). Nessun batching multi-request. Modello target: Llama-3.1-8B, 4-bit quantizzato. Draft: Llama-3.2-1B, 4-bit.

Misurazioni [MISURATO] + [PROIETTATO]. Speedup dichiarato: non riportato un singolo numero, ma *"significant throughput gains"* su *"most tested compute units"*. **Sotto Intel Core Ultra 7 CPU: throughput DECREASE** (regime compute-bound). **Sulla iGPU dello stesso chip: _significant gains due to memory bandwidth limitations_.** Modello teorico esplicito (calcolo dell'autore): *"For an 8B, 8-bit parameter model, this means transferring 8GB per token. On a system with 128GB/s memory bandwidth, this limits throughput to ~16 tokens per second, excluding other memory demands like KV-cache. Speculative decoding batches multiple tokens in one iteration, reading weights once to generate several tokens. Assuming a batch of 8 tokens for the same target model, this could theoretically boost throughput to ~128 tokens per second."* Etichetta: teoria **[PROIETTATO]** (assunzione batch=8); empirico **[MISURATO]** su Intel Core Ultra 7 CPU + iGPU. Acceptance length: non riportata numericamente. Memoria occupata dal drafter: *"draft model's weights, KV-cache, and context increase memory demands"* — qualitativo. Baseline esplicita: standard decoding dello stesso target (Llama-3.1-8B 4-bit) senza spec-decode.

Argomento teorico. Meccanismo: *"In standard autoregressive decoding, LLMs generate one token at a time, each requiring the model's weights to be read from RAM. [...] Speculative decoding batches multiple tokens in one iteration, reading weights once to generate several tokens."* Tassonomia

esplicita dell'autore: *"Memory Bound: which benefit from speculative decoding due to its impact on easing memory bandwidth"* vs *"Compute Bound: which perform worse performance with speculative decode due to the increased compute requirements"*. Questa è la split esatta del dossier.

Rilevanza single-user CPU: PARZIALE MA CRITICO. È l'unica fonte trovata che testa esplicitamente spec-decode su consumer CPU x86 (Intel Core Ultra 7) confrontandola con la iGPU sullo stesso package. Il risultato (CPU peggiora, iGPU guadagna) è l'evidenza empirica più rilevante per la domanda: **conferma che su CPU consumer il beneficio dipende dal rapporto compute/bandwidth**. Per il dossier (CPU memory-bound misurato) implica: il guadagno c'è se e solo se il decode è effettivamente bandwidth-bound, non compute-bound. L'autore raccomanda *graceful degradation* verso traditional decoding se il sistema è under compute pressure.

3.3 Speculative Decoding Meets Quantization

Citazione: [9] Zhang, Y.; Zhao, W.; Han, X.; Zhao, T.; Xu, W.; Cao, H.; Zhu, C., *Speculative Decoding Meets Quantization: Compatibility Evaluation and Hierarchical Framework Design*, arXiv:2505.22179v1, 28 maggio 2025. URL: <https://arxiv.org/abs/2505.22179> — codice: <https://github.com/AI9Stars/SpecMQuant>.

Regime sperimentale. Hardware: single NVIDIA A100 GPU (secondario: RTX 3090 in studio comparativo). Memory bandwidth: non riportata in GB/s, ma dichiarato il regime: *"the large-scale parameters and auto-regressive decoding make inference memory-bound, particularly on the single-batch inference of resource-constrained devices with model weights dominating memory bandwidth"*. Batch size: **single-batch inference esplicita**. Modello target: Llama-3-70B-Instruct, W4A16 (4-bit weight, 16-bit activation, Marlin kernel via vLLM). Draft: Llama-3-8B-Instruct (W4A16). Intermedio gerarchico: Llama-3-8B.

Misurazioni [MISURATO], baseline = Vanilla AR / Vanilla SP / EAGLE-2. Speedup dichiarato: **2.78×** su W4A16 Llama-3-70B su A100, media su WikiText2/GSM8K/HumanEval/MT-Bench. Speedup = 98.19 tok/s vs baseline AR 35.32 tok/s. **1.31×** over EAGLE-2 (SOTA spec-decode), **1.21×** over Vanilla SP. Acceptance length τ misurata (HierSpec, d1=3, d=6): **~4.62–6.58** a seconda del task (vs $\tau \approx 1$ di AR). EAGLE-2: $\tau \approx 3.31$ –4.23. Decoding speed misurato: 98.19 tok/s (HierSpec 6(3)) vs 35.32 tok/s (Vanilla AR) → **2.78×**. Memoria occupata dal drafter: W4A16 Llama-3-8B (~4 GB per il solo draft in 4-bit).

Argomento teorico. Meccanismo (dall'abstract): *"Speculative decoding mitigates the memory bandwidth bottleneck by verifying multiple tokens within a single forward pass, which increases computational effort."* — esplicita trade-off: spec-decode scambia compute extra per riduzione memory-access frequency. Finding critico del paper: *"experiments applying the advanced speculative decoding method EAGLE-2 to various quantized models reveal that the memory benefits from 4-bit weight quantization are diminished by the computational load from speculative decoding. Specifically, verifying a tree-style draft incurs significantly more time overhead than a single-token forward pass on 4-bit weight quantized models."* — cioè quando il target è troppo quantizzato e il weight loading è già leggero, spec-decode può smettere di essere conveniente perché il costo compute del tree-verification supera il risparmio bandwidth. Si noti però che questo avviene su A100 con

kernel Marlin molto ottimizzato: su CPU consumer senza VNNI il quadro è opposto (compute extra è a basso costo relativo, weight-load resta dominante).

Rilevanza single-user CPU: PARZIALE / INDIRETTO. Hardware è A100 GPU (non CPU consumer). Ma: (i) il framing teorico *"single-batch memory-bound, model weights dominating memory bandwidth"* è esattamente quello del dossier; (ii) la misurazione di accettazione $\tau \approx 4.6\text{--}6.6$ a draft length 6 è coerente con il soffitto $\approx 2\text{--}4\times$ del brief (anche se leggermente superiore grazie al tree drafting); (iii) il warning su *"compute load may diminish memory benefits"* è rilevante: avverte che su hardware con compute molto efficiente (es. A100 + Marlin) il vantaggio si assottiglia — su CPU consumer no-VNNI il peso del compute è maggiore ma resta subordinato al bandwidth.

3.4 Qualcomm Cloud AI 100 — Teoria bandwidth-amortization

Citazione: [10] Vaidhyathan, N. (Qualcomm), *How to Quadruple LLM Decoding Performance with Speculative Decoding (SpD) and Microscaling (MX) Formats on Qualcomm Cloud AI 100*, Qualcomm Developer Blog, marzo 2024. URL: <https://www.qualcomm.com/developer/blog/2024/03/quadruple-llm-decoding-performance-speculative-decoding-spd-microscaling-mx-formats>. Fonte vendor blog, non peer-reviewed, ma fonte primaria dell'argomento teorico bandwidth-amortization.

Regime sperimentale. Hardware: Qualcomm Cloud AI 100 (acceleratore inference, 3 varianti TDP: Standard 75W, Pro 75W, Ultra 150W). NON CPU consumer. Memory bandwidth: non riportata in GB/s; esplicitata come fattore limitante: *"Decode Phase is more DRAM-bandwidth constrained than computation constrained, because all the parameters of the model need to be fetched to process just one input token."* Batch size: **Batch Size 1 esplicito** (tabella configurazione esperimento). Modello target: CodeGen-1-7B mono (MXFP6) e Llama-2-7B-Chat-FT (MXFP6). Draft: 350M (CodeGen) e 115M (Llama-2). Speculation length $K=7$.

Misurazioni [MISURATO], baseline = no speculative decoding + FP16. Speedup dichiarato: $\sim 2\times$ da SpD da solo (greedy), $\sim 2\times$ da MX6 alone, $\sim 4\times$ **combined (SpD + MX6)** vs baseline FP16 no-SpD. *"Synergistically, both features together offer a multiplicative speedup of LLM decoding by a factor of ~ 4 ."* Acceptance length: non riportata numericamente (greedy SpD con $K=7$). Memoria occupata dal drafter: 350M FP16 (CodeGen) o 115M FP16 (Llama-2). Molto piccola relativamente al target.

Argomento teorico — formula first-order. Due passaggi chiave (testualmente): (i) Modello first-order del decode bound: *"To a first order approximation, the number of tokens that an LM can generate during the decode phase can be calculated as DRAM-Read-Bandwidth (bytes/sec) divided by the capacity of the model in bytes."* — questa è la formula esatta del regime memory-bound. (ii) Ammortizzazione: *"the decode phase can afford to do additional computation without impacting the latency. [...] SpD leverages the insight that it's faster to run a small model to generate K draft tokens and then verify them in one parallel pass through the large model."* Cioè: la forward pass singolo-token è già saturata dalla bandwidth; aggiungere compute (verifica di K draft nella stessa forward) non costa latency aggiuntiva finché si resta sotto il budget di compute lasciato libero dalla bandwidth-saturazione. Questo è esattamente il meccanismo di ammortamento del weight-stream.

Rilevanza single-user CPU: NO sul piano hardware, SÌ sul piano teorico. L'hardware è un acceleratore data-center, non CPU consumer. MA: (i) batch size = 1, single-user; (ii) la formula $\text{tok/s} = \text{DRAM-BW} / \text{model-size}$ è universalmente applicabile (valida anche per CPU consumer DDR4/DDR5); (iii) l'argomento *"compute extra è gratis in regime memory-bound"* è il pilastro teorico del dossier.

3.5 Sintesi — Stato dell'evidenza

Evidenza teorica bandwidth-amortization: FORTE E CONVERGENTE. Cinque fonti indipendenti formulano esplicitamente l'argomento che spec-decode ammortizza il weight-stream, non il compute: (1) Qualcomm Cloud AI 100 blog fornisce la formula first-order $\text{tok/s} = \text{DRAM-BW} / \text{model-size}$; (2) Speculative Decoding Meets Quantization: *"speculative decoding mitigates the memory bandwidth bottleneck by verifying multiple tokens within a single forward pass"*; (3) Ghidorah: *"Since the sequential decoding approach is memory-bandwidth-bound, its execution time is similar to that of Medusa"*; (4) STree modella $\text{Wall Time} \propto t/\tau_{\text{cont}} = kN + C$ dove C è il costo di loading dei pesi (costante rispetto a N) — è la formalizzazione matematica dell'ammortamento: quando C domina su kN , $\tau > 1$ riduce linearmente Wall Time; (5) SpecMamba (arXiv 2509.19873, SSM-specifico): *"Despite Mamba's impressive efficiency gains, its autoregressive generation process remains memory-bound, leading to suboptimal computational utilization... speculative decoding has been proposed as a promising solution, effectively increasing computational density and alleviating bandwidth constraints."*

Evidenza empirica DIRETTA su single-user CPU consumer: PARZIALE / WEAK. OpenInfer blog (ago 2025) è l'unica fonte trovata che testa esplicitamente una consumer CPU x86 (Intel Core Ultra 7) con spec-decode (Llama-3.1-8B target + Llama-3.2-1B draft, 4-bit). Risultato: throughput DECREASE sulla CPU (compute-bound), throughput increase sulla iGPU (memory-bound). Conferma che il guadagno è condizionale al regime. Ghidorah (2025) testa edge ARM CPU + GPU (Jetson NX) in regime memory-bandwidth-bound dichiarato, single-sample, $7.6\times$ speedup (di cui $3.27\times$ algoritmico puro). EdgeLLM (Xu et al., IEEE TMC 2024, DOI 10.1109/TMC.2024.3513457) è citato in SLED e Lever come spec-decode su flash-backed LLM mobile (flash-backed implica weight-stream-bound per costruzione), ma PDF non estraibile in questa sessione.

Evidenza INDIRETTA su SSM target: SOLIDA. SpecMamba (arXiv 2509.19873) — Mamba target + spec-decode su FPGA, $2.27\times$ over GPU baseline, $2.85\times$ over prior FPGA, framing esplicito memory-bound per SSM. STree (arXiv 2505.14969) — hybrid SSM/Transformer, MambaInLlama-8B, $1.69\text{--}1.93\times$ over autoregressive su RTX 3090; $1.21\text{--}1.62\times$ su H100. Notevole: *"the memory bandwidth of H100 GPU is bigger than that of RTX3090, leading to a smaller bottleneck in memory transfer... as the model size increases and the memory bandwidth bottleneck due to the model weight transfer becomes more significant, we will still be able to show improvement"* — conferma che il guadagno di spec-decode cresce al crescere del grado di memory-bound. Mamba Drafters (arXiv 2506.01206) — Mamba come drafter (non target); *"By leveraging the linear structure of SSMs, our approach avoids the quadratic complexity inherent in traditional Transformer-based methods, enabling faster drafting and lower memory usage"*. Snakes-and-Ladders (Wu et al., PMLR v262, 2024) — fondativo per spec-decode su SSM; introduce Activation

Replay e Joint Attainment and Advancement per gestire il backtracking dello stato SSM (problema specifico dei modelli selettivi tipo Mamba, rilevante per il dossier).

Evidenza INDIRETTA su small-batch on-device: SOLIDA. Staged Speculative Decoding (Spector & Re, arXiv 2308.04623, 2023) esplicita "*small-batch, on-device scenarios*" e "*low arithmetic intensity of small-batch inference*". $3.16\times$ su GPT-2-L 762M. È la fonte fondativa del framing "*small-batch = memory-bound = spec-decode conveniente*".

Numeri di ceiling e coerenza con il brief. I numeri di acceptance length / speedup algoritmico puro (escludendo parallel-speedup eterogeneo) nei paper memory-bound sono coerenti con il soffitto pre-registrato $\approx 2-4\times$: Ghidora algoritmico puro $3.27\times$ (verification width 16, MBPP); SpecDec-Quant HierSpec $2.78\times$ (Llama-3-70B W4A16, A100); STree Vanilla SD $1.69-1.93\times$ (MambaInLlama-8B, RTX 3090); STree $1.36-1.62\times$; SpecMamba $2.27\times$ over GPU baseline; Staged Spec Dec $3.16\times$ (GPT-2-L); Qualcomm SpD alone $\sim 2\times$ (Cloud AI 100, batch=1). Range tipico acceptance length τ : $2.0-3.7$ per verification width $4-32$ (Ghidora, MBPP/HumanEval/GSM8K). **Convergenza: ceiling algoritmico puro nei paper memory-bound su hardware eterogeneo è $2-3.3\times$, con $\tau \approx 2-3.7$.** Il soffitto onesto del brief $\approx 2-4\times$ è quindi **allineato o leggermente conservativo** rispetto all'evidenza pubblicata.

GAP DI EVIDENZA — DICHIARAZIONE ESPLICITA. Nessun paper peer-reviewed misura spec-decode / block-decode su **CPU consumer x86 single-thread single-user con modello SSM selettivo (Mamba-style)**. L'evidenza diretta sullo scenario esatto del dossier è **ASSENTE**. Cosa c'è (evidenza parziale cumulata): (1) spec-decode su consumer x86 single-user con LLM Transformer (OpenInfer, blog vendor) — parziale, qualitativo; (2) spec-decode su edge ARM CPU+GPU single-sample con LLM Transformer (Ghidora) — quantitativo, hardware vicino; (3) spec-decode su SSM/Mamba target (SpecMamba FPGA, STree GPU) — SSM target, hardware non CPU; (4) spec-decode + quantized target su A100 single-batch (SpecDec-Quant) — framing teorico identico, hardware non CPU; (5) spec-decode su small-batch on-device (Staged Spec Dec) — framing fondativo, hardware non documentato in abstract; (6) spec-decode su flash-backed mobile LLM (EdgeLLM) — flash-backed implica weight-stream-bound, hardware mobile, PDF non estraibile. **Cosa manca:** misura diretta di tok/s e acceptance length su CPU x86 consumer (Zen 2 / Intel Core) con modello SSM; misura diretta single-thread (non multi-core parallel) per validare il ceiling $\approx 2-4\times$ nel regime più pessimistico; studio di spec-decode con MLP ternario (gated-dReLU) come target — nessun paper trovato, questa è una zona bianca.

4. Domanda 4 — Self-speculative senza modello separato

Sei paper rilevanti identificati, coprenti le famiglie: LayerSkip (Meta, ACL 2024), CALM (Google, NeurIPS 2022), SWIFT (Huawei, ICLR 2025), Medusa (Princeton, ICML 2024), EAGLE/EAGLE-2 (2024), QuantSpec (Apple/Berkeley, ICML 2025). **Finding critico:** NESSUNO dei paper nella letteratura self-speculative fornisce evidenza sperimentale su modelli $<1\text{B}$ parametri, su CPU, o su architetture SSM. Tutti i claim di speedup sono proiezioni al di fuori del regime target del dossier ($\sim 5\text{M}$ parametri, SSM, CPU single-user). La taglia più piccola testata è Llama-3.2-1B (LayerSkip).

Correzioni fattuali al brief. (a) "*Apple Self-Speculative Decoding (2024)*": in realtà Apple pubblica "*QuantSpec: Self-Speculative Decoding...*" in ICML 2025 (luglio 2025), autori UC Berkeley+Apple. Non esiste un paper Apple 2024 con titolo *Self-Speculative Decoding*. (b) "*Bertrand 2024 Inference with Reference*": l'autore reale è **Nan Yang et al. (2023)**, arXiv:2304.04487. Il paper (LLMA) usa testo di riferimento esterno come draft, NON è self-speculative nel senso stretto (richiede retrieved document) — pertanto non è incluso nelle estrazioni principali. (c) "*Hash-based draft (SpecDec, SpecExec)*": SpecExec non usa hashing; usa *cache tree* di token più probabili. SpecDec non è un paper specifico identificato con questo nome; probabilmente riferito a *Block Verification Accelerates Speculative Decoding* (arXiv:2403.10444) che è una tecnica di verifica a blocchi.

4.1 LayerSkip — Enabling Early Exit and Self-Speculative Decoding

Citazione: [11] Elhoushi, M. et al. (Meta AI), *LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding*, ACL 2024 (Long). arXiv:2404.16710. URL: <https://arxiv.org/abs/2404.16710> — codice: <https://github.com/facebookresearch/LayerSkip>.

Metodo esatto. Self-speculative puro: nessun modello drafter separato. Il draft si ottiene uscendo (*early exit*) dal modello target a un layer intermedio $N < K$. La verifica è eseguita calcolando i layer rimanenti $K - N$ dello stesso modello. Verificatore = modello completo (stesso backbone, layer profondi). Condivide pesi, KV cache e attivazioni tra fase di draft e fase di verifica. Il draft viene generato auto-regressivamente fino a un exit-layer, poi si usa un LM head (unembedding) applicato alle hidden states del layer intermedio.

Costo retrofit. Co-training obbligatorio. La recipe di training comprende: (i) layer dropout con rate crescente (basso per layer iniziali, alto per layer finali), (ii) early-exit loss condivisa su tutti i transformer layers. Testato in 4 modalità: pretraining from scratch, continual pretraining, finetuning su dominio, finetuning su task. **Non zero-shot.** Applicare `assistant_early_exit=N` a un modello non-LayerSkip-trained NON produce speedup (il blog HF lo dichiara esplicitamente). Esistono checkpoint pubblici: facebook/layer-skip-llama2-7B, layer-skip-llama2-13B, layer-skip-llama2-70B, layer-skip-llama3-8B, layer-skip-llama3.2-1B.

Accettazioni misurate [MISURATO], baseline = autoregressive greedy decoding sullo stesso modello. Taglia minima testata: **Llama-3.2 1B** (Early Exit @ Layer 4) — summarization: efficiency **1.80×**. Taglia media: Llama2-7B (varie configurazioni); Llama3-8B; Llama2-13B (Early Exit @ Layer 8: **1.746×**; @ Layer 4: 0.995×). Taglia grande: Llama2-70B (Early Exit @ Layer 10, summarization): **2.06×**. Domini: summarization (CNN/DM), coding (HumanEval-style), TOPv2 semantic parsing. Numeri headline del paper: fino a **2.16×** su **CNN/DM summarization**, **1.82×** su **coding**, **2.0×** su **TOPv2**. Confronto anche vs traditional speculative decoding con OpenELM-270M/450M come drafter (1.57× / 1.32× su Llama2-13B).

Applicabilità a ~5M parametri. Nessuna evidenza diretta su modelli <1B. Il modello più piccolo testato è Llama-3.2-1B (1B parametri, 16 layer). Modelli ~5M sono 200× più piccoli. Nessuna evidenza su modelli <100M. Nessuna evidenza su SSM selettivi (solo Llama transformer decoder-only).

Regime del claim. Hardware: GPU (A100 80GB single-GPU; 70B su nodo 8×A100). Batch: single-stream (non dichiarato esplicitamente batched serving; benchmark in stile single-user latency). Overhead: overhead di draft ridotto grazie a condivisione KV-cache; la fase di draft usa gli stessi pesi senza memoria aggiuntiva.

Rilevanza single-user CPU: PARZIALE. Il design *shared weights / shared KV cache* è in linea con regime single-user, ma tutti i numeri sono su A100 GPU. Su CPU memory-bound il vincolo dominante è la bandwidth dei pesi non compute, e l'early-exit può ridurre FLOPs senza ridurre l'accesso ai pesi dei layer skipped (se i pesi vanno comunque caricati). Da verificare empiricamente — il paper NON lo testa.

Gap di evidenza. Misurazioni su modelli <100M; misurazioni su CPU; misurazioni su SSM/Mamba (i layer dropout su SSM selettivi non sono studiati); speedup reale in regime memory-bound (dove FLOPs saved ≠ tempo saved); trade-off exit-layer per modelli con pochi layer (<8).

4.2 CALM — Confident Adaptive Language Modeling

Citazione: [12] Schuster, T.; Fisch, A.; Gupta, J.; Dehghani, M.; Dua, D.; Hou, H.; Zettlemoyer, L.; Joshi, M.; Peters, M.E.; Cohen, W.W.; Borgeaud, S. (Google), *Confident Adaptive Language Modeling*, NeurIPS 2022 (Oral). arXiv:2207.07061. URL: <https://arxiv.org/abs/2207.07061>.

Metodo esatto. Early-exit puro al livello token. Per ogni token, il modello decide se uscire a un layer $N < K$ basandosi su una misura di confidenza. Non è *self-speculative decoding* nel senso moderno (no draft+verify a span): è adaptive compute per token. Le decisioni di exit sono locali; il paper affronta il problema di *attend back to missing hidden representations* (rappresentazioni dei layer successivi mancanti per i token che sono usciti prima). Verificatore: non separato, è lo stesso modello che prosegue per i token non-confident. Misura di confidenza: max-softmax, margin, predicted entropy (paper fa analisi teorica e empirica).

Costo retrofit. Fine-tuning leggero post-hoc. Il modello base (T5/encoder-decoder e decoder-only) viene dotato di classificatori di confidenza (linear head) e le soglie di exit vengono calibrate su un set di validazione. NON richiede pretraining from scratch con recipe speciale come LayerSkip. Tuttavia, i modelli non sono *zero-shot*: serve calibration e alcuni classificatori.

Accettazioni misurate [MISURATO]. Speedup headline: fino a 3× (potential speedup, dichiarato dall'abstract). Task: CNN/DM summarization, SQuAD question answering, WMT-En-Fr translation. Modelli: T5-base (220M), T5-large (770M) — encoder-decoder. Anche decoder-only. Baseline: modello completo senza early exit. Metriche di qualità mantenute (lossless entro epsilon garantito teoricamente). Nessuna misurazione su modelli >1B dichiarata in questo paper.

Applicabilità a ~5M parametri. Evidenza parziale su modelli piccoli (220M T5-base è il più piccolo) ma comunque 40× più grande di 5M. Nessuna evidenza su <100M e nessuna su ~5M. Encoder-decoder (T5) è una famiglia architetturale diversa da SSM selettivo.

Regime del claim. Hardware: TPU (Google interno) — non specificato esplicitamente GPU/CPU ma le eval sono su TPU v4. Batch: batched inference (le valutazioni sono su dataset con batching). Overhead: calibrazione di confidenza per-step; overhead minimo (linear head).

Rilevanza single-user CPU: PARZIALE. L'idea di adaptive compute per token è ortogonale a hardware; in regime memory-bound CPU, però, l'early-exit per singolo token può NON dare speedup se i pesi dei layer skipped devono comunque essere caricati in cache. Il paper non lo testa su CPU.

Gap di evidenza. Misurazioni su CPU; misurazioni su decoder-only con <100M; misurazioni su SSM; effetto del batching (il paper nota che il batching erode i guadagni dell'early-exit, con correlazione negativa all'aumentare del batch) — non chiaro come si comporti in single-user.

4.3 SWIFT — On-the-Fly Self-Speculative Decoding

Citazione: [13] Xia, H.; Yang, Z.; Qi, S.; Liu, Y.; Ge, T.; Wang, P.; Lin, Y.; Sui, Z. (Huawei; Peking University), *SWIFT: On-the-Fly Self-Speculative Decoding for LLM Inference Acceleration*, ICLR 2025. arXiv:2410.06916. URL: <https://arxiv.org/abs/2410.06916>.

Metodo esatto. Layer-skip plug-and-play: **skip di layer intermedi del modello target** (non early-exit al layer $N < K$, ma rimozione di subset di layer intermedi). Il modello stesso, con alcuni layer rimossi, è il drafter. Layer-skipping è task-specific e adattivo: SWIFT seleziona on-the-fly quali layer saltare in base all'input. Verificatore = modello completo (target LLM originale). Permette drafting di span multipli come in speculative decoding standard, ma il drafter è una versione *compressa* del target.

Costo retrofit. Zero-shot. "*SWIFT does not require auxiliary models or additional training, making it a plug-and-play solution for accelerating LLM inference across diverse input data streams*" (abstract). La selezione dei layer da saltare avviene al runtime, basata su analisi di sparsity task-specific.

Accettazioni misurate [MISURATO]. Speedup: **1.3×–1.6×** preservando la distribuzione di generazione. Modelli testati: *wide range of models* (dettagli esatti non nell'abstract). Task: *diverse downstream tasks*. Baseline: autoregressive standard + altri metodi self-speculative.

Applicabilità a ~5M parametri. Nessuna evidenza diretta su <100M. L'abstract non specifica taglie minime, ma il contesto ICLR 2025 e LLM suggerisce 1B+. L'approccio zero-shot è in linea con vincoli di retrofit di una fondazione ~5M (nessun training aggiuntivo). Però l'efficacia su modelli con pochi layer non è misurata: se un modello ha solo 4-8 layer, lo spazio di layer-skipabili è minimo.

Regime del claim. Hardware: non dichiarato nell'abstract, verosimilmente GPU A100/H100 (standard per ICLR LLM paper). Batch: non dichiarato; *diverse input data streams* suggerisce single-/low-batch. Overhead: il drafter on-the-fly seleziona layer skip; l'overhead di selezione va misurato nel full paper.

Rilevanza single-user CPU: PARZIALE / SÌ (teorico). Il design plug-and-play senza training è attraente per single-user CPU memory-bound. Su CPU, saltare layer riduce sia FLOPs che accessi ai pesi — a differenza di LayerSkip su GPU dove domina compute. Quindi su CPU memory-bound, SWIFT potrebbe essere piuttosto efficace. **Ma non è misurato.**

Gap di evidenza. Misurazioni su CPU; misurazioni su modelli <100M; misurazioni su SSM selettivi; effetto del numero di layer totali del modello (modelli con <8 layer); overhead di selezione dinamica in single-user.

4.4 Medusa — Multiple Decoding Heads

Citazione: [14] Cai, T.; Li, Y.; Geng, Z.; Peng, H.; Lee, J.D.; Chen, D.; Dao, T. (Princeton; UIUC), *Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads*, ICML 2024. arXiv:2401.10774. URL: <https://arxiv.org/abs/2401.10774> — codice: <https://github.com/FasterDecoding/Medusa>.

Metodo esatto. Multi-head drafting sullo stesso backbone: aggiunge *extra decoding heads* (uno per ogni posizione futura) collegati agli hidden states del layer finale del modello target. Ogni head predice il token a $t+1$, $t+2$, ..., $t+K$. Tree-based attention: costruisce albero di candidati e verifica in parallelo in un solo forward del target. Backbone resta (in Medusa-1) invariato; in Medusa-2 backbone viene joint-trained con le head. Verificatore = backbone completo (stesso modello). NON usa un modello drafter separato (le head sono shallow linear/MLP), quindi rientra in *self-speculative per backbone condiviso* ma con parametri aggiuntivi.

Costo retrofit. Fine-tuning leggero post-hoc (Medusa-1): head-only, backbone frozen. Dichiarata *lossless inference acceleration*. **Joint training (Medusa-2):** backbone + head co-fine-tuned con recipe speciale per preservare capacità. Migliore speedup ma costo superiore. **Self-distillation** disponibile quando non si hanno dati di training. NON zero-shot.

Accettazioni misurate [MISURATO]. Medusa-1: oltre 2.2× speedup lossless. Medusa-2: 2.3×–3.6× speedup. Modelli testati: Vicuna-7B/13B, Llama-2-7B/13B/70B, Zephyr-7B, FairSeq 7B/13B/70B. Taglia minima: 7B. Nessuna evidenza <1B. Baseline: autoregressive greedy. Metriche qualità: HF HumanEval, AlpacaEval, MT-Bench.

Applicabilità a ~5M parametri. Nessuna evidenza diretta su <1B. Testato su 7B-70B. Per modelli ~5M, le head aggiuntive hanno un peso relativo non trascurabile e il tree-attention aggiunge overhead. Plausibile ma non misurato.

Regime del claim. Hardware: GPU A100 (le experiment section nel paper usa A100/H100). Batch: single-stream (ottimizzato per latency, dichiarato come primary use case). Overhead: ogni step aggiunge K forward delle head (ma le head sono shallow, MLP, su backbone già calcolato); l'overhead è ~1.5-2× compute per step ma riduce #step totali.

Rilevanza single-user CPU: PARZIALE. Single-stream va bene, ma su CPU memory-bound l'overhead di calcolo delle head aggiuntive (anche se shallow) può erodere il gain se i pesi del backbone dominano. Inoltre tree-attention richiede modifiche alla KV-cache e può essere inefficiente su CPU se non ottimizzato. **Non misurato.**

Gap di evidenza. Misurazioni su CPU; misurazioni su <1B; misurazioni su SSM; overhead delle head in regime memory-bound.

4.5 EAGLE / EAGLE-2 — Feature-level Drafter

Citazione: [15] Li, Y.; Wei, F.; Zhang, C.; Zhang, H., *EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty*, arXiv:2401.15077. URL: <https://arxiv.org/abs/2401.15077> — codice: <https://github.com/SafeAILab/EAGLE>. [16] Li, Y.; Wei, F.; Zhao, J.; Zhang, C.; Zhang, H.,

EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees, arXiv:2406.16858.
URL: <https://arxiv.org/abs/2406.16858>.

Metodo esatto. Drafter feature-level, non vocabulary-level. EAGLE opera l'autoregressione al livello di feature (second-to-top-layer hidden state) anziché al livello di token. Il drafter è un piccolo modello (1 layer transformer + proiezione) che predice l'hidden state del target al prossimo step. Input del drafter: (i) hidden state del target al passo t , (ii) token sequence shifted di uno (risolve l'incertezza feature-level). Auto-regressivo nel feature space. Verificatore = target model completo. Tree-attention per verifica parallela. EAGLE-2 aggiunge **context-aware dynamic draft tree** (albero di draft con struttura adattiva basata su confidence del drafter, che è ben calibrato). Il drafter è tecnicamente un modello separato, ma condivide il representation space del target (feature-level, non vocabolario) ed è lightweight (1 layer).

Costo retrofit. Fine-tuning drafter post-hoc obbligatorio. Il drafter viene allenato su un dataset raccolto dal target (richiede forward pass del target per raccogliere features). NON zero-shot. NON co-training del target. EAGLE-2 non aggiunge costo di training rispetto a EAGLE-1.

Accettazioni misurate [MISURATO], baseline = autoregressive. EAGLE: 2.7×–3.5× latency speedup su LLaMA2-Chat 70B; throughput raddoppiato. Modelli testati: tutti Vicuna series (7B/13B), LLaMA2-Chat 7B/13B/70B, Mixtral 8×7B (MoE). Task: dialogue, code generation, mathematical reasoning, instruction following. **EAGLE-2: 3.05×–4.26×**, 20%-40% più veloce di EAGLE-1; lossless. Taglia minima: 7B. Nessuna evidenza <1B.

Applicabilità a ~5M parametri. Nessuna evidenza diretta su <1B. Il drafter feature-level richiede hidden states del target accessibili; su un SSM selettivo il *second-to-top-layer feature* non ha un equivalente diretto (architettura diversa). L'addestramento del drafter su una fondazione ~5M è plausibile ma il costo di training aggiuntivo è non-trascurabile.

Regime del claim. Hardware: GPU A100/H100. Batch: single-stream (low-batch / latency-ottimizzato). Overhead: drafter 1-layer forward per ogni step di draft. Minimo rispetto al target ma richiede passaggio di dati tra target hidden state e drafter.

Rilevanza single-user CPU: PARZIALE / BASSO. Il drafter separato anche lightweight richiede parametri aggiuntivi in memoria. Su CPU single-user memory-bound, il drafter 1-layer deve stare in cache o essere caricato. Non misurato. Il tree-attention è inefficiente su CPU senza batching.

Gap di evidenza. Misurazioni su CPU; misurazioni su <1B; misurazioni su SSM (i *feature-level* non si mappano direttamente a SSM selettivi); overhead del drafter in regime memory-bound.

4.6 QuantSpec — Self-Speculative with Hierarchical Quantized KV Cache

Citazione: [17] Tiwari, R.; Xi, H.; Tomar, A.; Hooper, C.; Kim, S.; Horton, M.; Najibi, M.; Mahoney, M.W.; Keutzer, K.; Gholami, A. (UC Berkeley; Apple), *QuantSpec: Self-Speculative Decoding with Hierarchical Quantized KV Cache*, ICML 2025. URL: <https://machinelearning.apple.com/research/quantspec>. Pubblicato luglio 2025.

Metodo esatto. Self-speculative: il draft model condivide architettura e pesi con il target, ma usa **KV cache gerarchicamente quantizzata a 4-bit** e **pesi quantizzati a 4-bit** per accelerare la fase di draft. Verificatore = target model a piena precisione (FP16/BF16). Specifico per long-context: la motivazione è che KV cache è il bottleneck in long-context inference su edge devices.

Costo retrofit. Non esplicitato nell'abstract; è implicito che il draft reusa i pesi del target in versione quantizzata (4-bit). Non richiede drafter separato allenato, ma richiede infrastruttura di quantizzazione gerarchica.

Accettazioni misurate [MISURATO]. Acceptance rate >90%. End-to-end speedup: ~2.5×. Memory reduction: ~1.3× rispetto ad altre varianti self-speculative con sparse KV cache. Confronto esplicito vs altri self-speculative methods con sparse KV cache (baseline). Taglie modelli: non dichiarate nell'abstract ma il contesto Apple edge/long-context suggerisce Llama-class 7B-13B. Non misurato <1B.

Applicabilità a ~5M parametri. Nessuna evidenza diretta. Long-context + edge device è compatibile con SSM selettivo in linea di principio, ma il meccanismo di quantizzazione gerarchica del KV cache non è validato su modelli tiny. Su SSM selettivi il KV cache è diverso (Mamba ha state ricorrente, non KV cache tradizionale).

Regime del claim. Hardware: GPU (edge devices con long-context). Niente CPU dichiarata. Batch: long-context single-user / low-batch. Overhead: doppia rappresentazione dei pesi (full + 4-bit) per target e drafter.

Rilevanza single-user CPU: PARZIALE. Long-context single-user è rilevante. Però l'accelerazione dipende dal fatto che 4-bit è più veloce del full-precision sullo stesso hardware — vero su GPU con tensor-core int4, ma su CPU l'efficienza di int4 inference dipende dal backend (AVX-512, NEON). Non misurato.

Gap di evidenza. Misurazioni su CPU; misurazioni su <1B; misurazioni su SSM selettivi (KV cache concetto non applicabile direttamente); overhead di doppia rappresentazione pesi in memoria CPU.

5. Tabella riassuntiva metodo × dimensioni

La tabella sintetizza i 17 metodi estratti nelle sezioni 1-4. Ogni numero è etichettato **[MISURATO]** o **[PROIETTATO]** e associato alla rispettiva baseline. La colonna *Rilevanza single-user CPU* è il flag esplicito richiesto dal brief (sì/no/parziale). I dettagli per ogni metodo sono nelle note successive (sezione 6).

Tabella 1 — Metodi di decoding speculativo/a-blocchi: regime, costo, accettazione, compatibilità SSM, rilevanza CPU single-user.

Metodo	Regime del claim	Costo drafter/retrofit	Accettazione a / speedup misurata	Dipendenza dominio	Compatibilità à SSM	Rilevanza single-user CPU
DeepSeek-V3 MTP [1]	GPU H800, batched serving, MoE 671B/37B	Co-training obbligatorio ($\lambda=0.3$); retrofit NON testato	85-90% su 2° token [MISURATO]; 1.8× TPS [MISURATO] vs vanilla AR	Non disaggregato (various topics)	NO (Transformer MoE); self-spec via MTP head, non testato su SSM	NO
DSpark [2]	GPU datacenter (DGX Spark/GB10), live traffic batched, MoE 1.6T/49B	Retrofit SÌ (attachable, testato su Qwen/Gemma); co-training NO	60-85% per-user vs MTP-1 [MISURATO]; +30% accept len vs Eagle 3 [MISURATO]; 51-400% range corner-case [PROIETTATO terzi]	Non disaggregato (live traffic)	NO (Transformer MoE); metodo serial-head teoricamente trasferibile	NO
Mamba Drafters [3]	GPU H100, single-user, target Transformer 7B/6.9B, drafter Mamba-130M	From scratch (FineWeb-Edu+SFT); target-agnostic	HumanEval $\tau=3.91$ (3.16) / GSM-8K $\tau=4.57$ / XSum $\tau=4.55$ [MISURATO] vs no-drafter	Codice/math/prosa testati; long-context stabile	PARZIALE: drafter SSM, verificatore Transformer (no SSM verifier)	NO
Snakes & Ladders [4]	GPU batched, target Mamba 1.3B/13B, drafter n-gram	Drafter n-gram, costo zero; nessun retrofit	6 token al costo di 1.47× singolo token; wall-clock 1.82× [MISURATO] vs SSM AR	Non riportato (three benchmarks, non specificati)	SÌ: verificatore Mamba; Activation Replay per stato SSM	PARZIALE (concettuale, no CPU)
STree [5]	GPU RTX 3090, single-user, target MambaInLlama-8B (50% ibrido), drafter Mamba2-130M distillato	Retrofit: distillazione 2-layer dal target, 48k step	MT-Bench $\tau=2.47$ (1.74×); HumanEval $\tau=2.79$ (1.95×); GSM-8K $\tau=2.83$ (1.98×) [MISURATO] vs AR	Prosa/codice/math testati (3 benchmark)	SÌ: verificatore Mamba2/SSD ibrido; TreeScan kernel + Activation Replay	PARZIALE (single-user, no CPU)

Metodo	Regime del claim	Costo drafter/retrofit	Accettazione a / speedup misurata	Dipendenza dominio	Compatibilità a SSM	Rilevanza single-user CPU
Mamba in the Llama [6]	GPU RTX 3090, single-user, target Mamba2 hybrid 7B/8B, drafter Transformer 2/4-layer	Retrofit: drafter Transformer fine-tune 3 epoche OpenHermes2 .5	Speedup 1.58-1.88× [MISURATO] vs target senza spec-decode	Non riportato (chat continuations OpenHermes2 .5)	Sì: verificatore Mamba2/SSD hybrid; MultiStep kernel + snapshot h_j	PARZIALE (single-user, no CPU; naive impl peggio su H100)
Ghidorah [7]	Jetson NX (ARM+GPU edge, unified memory), single-sample batch=1, target Vicuna-7B Medusa-5head	Medusa head (same backbone); no external drafter	7.6× total [MISURATO] = 3.27× algoritmico × 2.31× parallel; $\tau=3.27$ MBPP width=16 [MISURATO]	Codice/math/prosa testati (MT-Bench/GSM-8K/MBPP/HumanEval)	NO (Transformer target)	PARZIALE (single-user + memory-bound dichiarato, no x86 CPU, no SSM)
OpenInfer blog [8]	Intel Core Ultra 7 CPU + iGPU, single-user, Llama-3.1-8B 4-bit + Llama-3.2-1B 4-bit draft	Draft model esterno (Llama-3.2-1B 4-bit)	CPU: throughput DECREASE [MISURATO]; iGPU: significant gains [MISURATO]; teoria 128GB/s → 8GB/token → 16 tok/s baseline [PROIETTATO]	Non disaggregato	NO (Transformer target)	PARZIALE (unica fonte diretta su consumer x86 CPU)
SpecDec-Quant [9]	GPU A100, single-batch, Llama-3-70B W4A16 + Llama-3-8B W4A16 draft	Retrofit: gerarchico (drafter 8B + intermedio 8B + target 70B)	2.78× speedup; $\tau=4.62-6.58$ (HierSpec d1=3,d=6) [MISURATO] vs Vanilla AR 35.32 tok/s	WikiText2/ GSM8K/ HumanEval/ MT-Bench (media)	NO (Transformer target)	PARZIALE (framing teorico identico, hardware no CPU)
Qualcomm Cloud AI 100 [10]	Qualcomm AI 100 accelerator, batch=1, CodeGen-7B/ Llama-2-7B MXFP6, draft 350M/115M K=7	Draft esterno small (350M / 115M)	~2× SpD alone; ~2× MX6 alone; ~4× combined [MISURATO] vs FP16 no-SpD	Non disaggregato (CodeGen/Llama-2)	NO (Transformer target)	NO hardware, SÌ teoria (formula tok/s = DRAM-BW / model-size)

Metodo	Regime del claim	Costo drafter/retrofit	Accettazione a / speedup misurata	Dipendenza dominio	Compatibilità a SSM	Rilevanza single-user CPU
LayerSkip [11]	GPU A100, single-stream, Llama2-7B/13B/70B + Llama3-8B/3.2-1B	Co-training obbligatorio (layer dropout + early-exit loss); non zero-shot	1.80× su Llama-3.2-1B (1B taglia min); 2.16× CNN/DM; 1.82× coding; 2.06× Llama2-70B [MISURATO] vs AR	Summarization/coding/TOPv2; non reasoning esplicito	NO (Transformer); layer dropout su SSM non studiato	PARZIALE (shared weights, no CPU; FLOPs saved ≠ time saved se memory-bound)
CALM [12]	TPU v4, batched, T5-base 220M / T5-large 770M (encoder-decoder)	Fine-tuning leggero (calibration + linear head); no co-training	Fino a 3× [MISURATO] (potential speedup abstract); lossless entro epsilon	Summarization/QA/translation; non codice	NO (T5 encoder-decoder)	PARZIALE (early-exit per token, no CPU; layer skipped pesi comunque caricati)
SWIFT [13]	GPU A100/H100 (presunto), single-/low-batch, LLM 1B+ (range non dichiarato)	Zero-shot plug-and-play (no training, no auxiliary model)	1.3-1.6× [MISURATO] preservando distribuzione	Diverse downstream tasks (non disaggregato)	NO (Transformer); layer-skip su SSM non studiato	PARZIALE / SÌ teorico (zero-shot, skippare layer riduce accessi pesi su CPU)
Medusa [14]	GPU A100/H100, single-stream, Vicuna/Llama-2/Zephyr 7B-70B	Fine-tuning head-only (Medusa-1) o joint (Medusa-2); no zero-shot	Medusa-1: >2.2× lossless; Medusa-2: 2.3-3.6× [MISURATO] vs AR	HumanEval/AlpacaEval/MT-Bench (multi-dominio)	NO (Transformer); head su SSM non testato	PARZIALE (single-stream, no CPU; overhead head in memory-bound non misurato)
EAGLE / EAGLE-2 [15][16]	GPU A100/H100, single-stream, Vicuna/Llama-2-Chat/Mixtral 7B-70B	Fine-tuning drafter post-hoc (1-layer transformer); no zero-shot, no co-training	EAGLE: 2.7-3.5×; EAGLE-2: 3.05-4.26× [MISURATO] vs AR; lossless	Dialogue/code/math reasoning esplicito	NO (Transformer); feature-level non mappabile a SSM	PARZIALE / BASSO (drafter separato, no CPU, tree-attention inefficiente su CPU)
QuantSpec [17]	GPU edge (long-context), single-user/low-batch, Llama-class 7B-13B (presunto)	Quantizzazione gerarchica 4-bit (no drafter separato)	~2.5× speedup; acceptance >90% [MISURATO] vs sparse KV cache baseline	Long-context (non disaggregato)	NO (KV cache concetto non applicabile a SSM selettivo)	PARZIALE (long-context single-user, no CPU; int4 su CPU dipende da backend)

Fonti: [1]-[17], vedi Bibliografia. τ = acceptance length; AR = autoregressive baseline.

6. Note per metodo

Note di contesto (2-4 frasi per metodo) che integrano la tabella: baseline, regime del claim, caveat. **Niente raccomandazioni architetturali** — la decisione di design è a valle, con i dati Cat-A di Phase 62.

Domanda 1 — DeepSeek

DeepSeek-V3 MTP [1]. Baseline = vanilla autoregressive dello stesso modello. Il claim $1.8\times$ TPS è [MISURATO] in regime GPU H800 batched serving su MoE 671B/37B. L'MTP è co-trained da zero (loss L_{MTP}^k con $\lambda=0.3-0.375$); retrofit su checkpoint esistente non testato. Acceptance 85-90% su 2° token NON disaggregata per dominio. Inapplicabile come retrofit a modelli SSM CPU esistenti.

DSpark [2]. Baseline = MTP-1 (production baseline di DeepSeek-V4). Claim 60-85% per-user [MISURATO] in regime GPU datacenter batched serving under live traffic su MoE 1.6T/49B o 284B/13B. Il range 51-400% NON appare nell'abstract del paper ed è attribuito a blog di terzi — da verificare nel full-text. Drafter attachable, retrofit esplicitamente supportato su Qwen/Gemma. Lossless (byte-for-byte identico al target). Verifica confidence-scheduled è pensata per alta concorrenza: il problema dell'allocazione batch capacity è inesistente in single-user.

Domanda 2 — SSM/Mamba

Mamba Drafters [3]. Baseline = no-drafter + EAGLE + external Transformer drafter. Misure [MISURATO] su GPU H100 single-user, target Transformer (Mistral-7B/Pythia-6.9B), drafter Mamba-130M. Verificatore NON è SSM: il caso *SSM come verificatore a-blocchi* non è testato in questo paper. Drafter target-agnostic (riusabile senza riaddestramento, vs EAGLE). Self-speculative Mamba (drafter + verificatore entrambi SSM con pesi condivisi) è future work dichiarato dagli autori.

Snakes & Ladders [4]. Baseline = inference SSM senza spec-decode. Misure [MISURATO] su GPU batched, target Mamba 1.3B/13B, drafter n-gram. **Primo paper a documentare Activation Replay per SSM:** cache di attivazioni intermedie del blocco + replay per ricomputare lo stato fino all'ultimo accettato. L'argomento chiave è *batched per fillare compute idle* — ortogonale al caso single-user. Scaling 1.3B → 13B mostra overhead relativo che scende da $1.98\times$ a $1.22\times$ (compute economy of scale).

STree [5]. Baseline = Vanilla SD + AR + FSS unrolled + Chunk Scan. Misure [MISURATO] su GPU RTX 3090 single-user, target MambaInLlama-8B (50% ibrido), drafter Mamba2-130M distillato. TreeScan kernel (sfrutta struttura diagonale di A di Mamba2) + Activation Replay. Speedup $1.36-1.98\times$ a seconda del dominio e temperatura. Memoria quasi costante al crescere dell'albero (8 → 8.3 GB per $M=2..5$, $N=16$) — FSS esplode a 14 GB poi OOM. Scaling overhead STree vs AR scende $2.04\times$ (2.3B) → $1.26\times$ (23B). Drafter distillato *early checkpoint* (48k step) — accept length probabilmente migliorabile.

Mamba in the Llama [6]. Baseline = target senza spec-decode. Misure [MISURATO] su GPU RTX 3090 single-user, target Mamba2 hybrid 7B/8B, drafter Transformer 2/4-layer. MultiStep kernel fused (recompute+decode+cache in una chiamata) con snapshot *hj pre-blocco*. *Assunzione esplicita del paper*: "decoding is bottlenecked by memory and not by compute"_. **Naive impl su H100 non dà speedup** (GEMM troppo veloce, overhead domina) → kernel fusion necessaria. Questo è il paper più rilevante per il dossier: il fallimento su H100 (regime meno memory-bound) suggerisce indirettamente che CPU (più memory-bound) sarebbe il caso migliore, ma non testato.

Domanda 3 — CPU memory-bound

Ghidorah [7]. Baseline = Sequential (GPU-only) + Medusa + Medusa+EM. Misure [MISURATO] su Jetson NX (ARM v8.2 + Volta GPU, unified memory, single-sample batch=1), target Vicuna-7B Medusa-5head. $7.6\times$ total = $3.27\times$ algoritmico $\times$ $2.31\times$ parallel. Tabella acceptance-length $\times$ verification-width (1-64) per MT-Bench/GSM-8K/MBPP/HumanEval è la più completa trovata. Osservazione chiave: sweet spot CPU = width 16, sweet spot GPU = width 64 (CPU satura prima). Single-sample + memory-bound dichiarato esplicitamente — ma hardware è edge ARM SoC, non CPU consumer x86.

OpenInfer blog [8]. Baseline = standard decoding dello stesso target. Fonte vendor blog (non peer-reviewed), ma **unica fonte diretta su consumer CPU x86** (Intel Core Ultra 7) con Llama-3.1-8B 4-bit + Llama-3.2-1B 4-bit draft. Risultato critico: throughput DECREASE su CPU (compute-bound), significant gains su iGPU (memory-bound) — split esatta del dossier. Modello teorico: $\text{tok/s} = \text{DRAM-BW} / \text{model-size} \rightarrow 8\text{GB/token su } 128\text{ GB/s} = 16\text{ tok/s baseline}$; con batch=8 teorico 128 tok/s [PROIETTATO]. Nessun numero di acceptance length.

SpecDec-Quant [9]. Baseline = Vanilla AR + Vanilla SP + EAGLE-2. Misure [MISURATO] su A100 single-batch, Llama-3-70B W4A16 + Llama-3-8B W4A16 draft gerarchico. $2.78\times$ speedup, $\tau=4.62-6.58$ (HierSpec d1=3, d=6). Framing teorico identico al dossier: *"single-batch inference of resource-constrained devices with model weights dominating memory bandwidth"*. **Warning critico**: *"memory benefits from 4-bit weight quantization are diminished by the computational load from speculative decoding"* — su A100 con kernel Marlin molto ottimizzato, il vantaggio si assottiglia. Su CPU consumer no-VNNI il quadro è opposto (compute extra è a basso costo relativo).

Qualcomm Cloud AI 100 [10]. Baseline = no-SpD + FP16. Misure [MISURATO] su acceleratore Qualcomm Cloud AI 100, batch=1, CodeGen-7B/Llama-2-7B MXFP6, draft 350M/115M K=7. $\sim 2\times$ SpD alone, $\sim 2\times$ MX6 alone, $\sim 4\times$ combined. Formula first-order esplicita: **`tok/s = DRAM-Read-BW / model-size`** — universalmente applicabile (anche CPU consumer DDR4/DDR5). Argomento teorico pilastro: *"the decode phase can afford to do additional computation without impacting the latency"* — è il meccanismo di ammortamento del weight-stream.

Domanda 4 — Self-speculative

LayerSkip [11]. Baseline = autoregressive greedy sullo stesso modello + traditional speculative decoding con OpenELM-270M/450M drafter. Misure [MISURATO] su GPU A100 single-stream, Llama2-7B/13B/70B + Llama3-8B/3.2-1B. Taglia minima testata: Llama-3.2-1B (1B parametri, 16

layer). Co-training obbligatorio (layer dropout + early-exit loss). **Non zero-shot**: applicare `assistant_early_exit=N` a un modello non-LayerSkip-trained NON produce speedup. Su CPU memory-bound il vincolo dominante è la bandwidth dei pesi non compute, e l'early-exit può ridurre FLOPs senza ridurre l'accesso ai pesi dei layer skipped — da verificare empiricamente.

CALM [12]. Baseline = modello completo senza early exit. Misure [MISURATO] su TPU v4 batched, T5-base 220M / T5-large 770M (encoder-decoder). Fino a 3× speedup. Fine-tuning leggero (calibration + linear head). Lossless entro epsilon garantito teoricamente. Il paper nota che il batching erode i guadagni dell'early-exit (correlazione negativa all'aumentare del batch) — non chiaro come si comporti in single-user. Encoder-decoder (T5) è famiglia architetturale diversa da SSM selettivo.

SWIFT [13]. Baseline = autoregressive standard + altri metodi self-speculative. Misure [MISURATO] su GPU A100/H100 (presunto), LLM 1B+ (range non dichiarato). **Zero-shot plug-and-play** — unico paper self-speculative senza training aggiuntivo. Speedup 1.3-1.6×. Su CPU, saltare layer riduce sia FLOPs che accessi ai pesi (a differenza di LayerSkip su GPU dove domina compute) — potrebbe essere efficace, ma non misurato. Su modelli con <8 layer, lo spazio di layer-skipabili è minimo: limite strutturale per una fondazione ~5M.

Medusa [14]. Baseline = autoregressive greedy. Misure [MISURATO] su GPU A100/H100 single-stream, Vicuna/Llama-2/Zephyr 7B-70B. Medusa-1: >2.2× lossless (head-only fine-tune, backbone frozen). Medusa-2: 2.3-3.6× (joint training backbone+head). Taglia minima 7B, nessuna evidenza <1B. Su CPU memory-bound, l'overhead di calcolo delle head aggiuntive (anche shallow) può erodere il gain se i pesi del backbone dominano. Tree-attention richiede modifiche alla KV-cache — inefficiente su CPU se non ottimizzato.

EAGLE / EAGLE-2 [15][16]. Baseline = autoregressive. Misure [MISURATO] su GPU A100/H100 single-stream, Vicuna/Llama-2-Chat/Mixtral 7B-70B. EAGLE: 2.7-3.5×; EAGLE-2: 3.05-4.26×, 20-40% più veloce di EAGLE-1. Drafter feature-level (1-layer transformer, opera su hidden state non vocabolario). Fine-tuning drafter post-hoc obbligatorio (NON zero-shot, NON co-training). Taglia minima 7B, nessuna evidenza <1B. Su SSM selettivi, il *second-to-top-layer feature* non ha equivalente diretto — architettura diversa. Tree-attention inefficiente su CPU senza batching.

QuantSpec [17]. Baseline = altri self-speculative methods con sparse KV cache. Misure [MISURATO] su GPU edge long-context single-user/low-batch, Llama-class 7B-13B (presunto). ~2.5× speedup, acceptance >90%. KV cache gerarchicamente quantizzata a 4-bit + pesi 4-bit per il draft; verificatore a piena precisione. Su SSM selettivi il *KV cache* è concetto non applicabile (Mamba ha state ricorrente, non KV cache tradizionale) — gap architetturale. Su CPU, l'efficienza di int4 inference dipende dal backend (AVX-512, NEON) — non misurato.

Sintesi trasversale — Gap di evidenza dichiarato

Per il regime target del dossier (CPU consumer x86 single-thread single-user, SSM selettivo stile Mamba-1 + SWA + MLP gated-dReLU ternario, memory-bandwidth-bound misurato), NESSUNO dei 17 metodi estratti fornisce evidenza sperimentale diretta. La copertura cumulata è:

- **Hardware:** 14/17 paper su GPU datacenter (H100/A100/3090/H800); 1/17 su edge ARM CPU+GPU (Ghidorah); 1/17 su acceleratore inference (Qualcomm); 1/17 su TPU (CALM); 1/17 blog vendor con consumer x86 CPU (OpenInfer, qualitativo). **Zero paper peer-reviewed su CPU consumer x86.**
- **Architettura target:** 11/17 Transformer; 4/17 SSM/Mamba (Mamba Drafters, Snakes & Ladders, STree, Mamba in the Llama) ma solo come verificatore in 3 casi; 1/17 T5 encoder-decoder (CALM); 1/17 FPGA SSM (SpecMamba, citato in sintesi). **Zero paper su SSM con MLP ternario gated-dReLU.**
- **Taglia modello target:** 13/17 su 7B+; 4/17 su 1-7B (Mamba Drafters 130M drafter con target 7B, Snakes & Ladders 1.3B-13B, LayerSkip 1B min, CALM 220M min). **Zero paper su <100M.** Zero paper su ~5M parametri (regime del dossier).
- **Regime batch:** 9/17 dichiarano single-user o single-stream; 6/17 dichiarano batched serving; 2/17 non dichiarato. Dei 9 single-user, tutti su GPU. **Zero paper single-user su CPU.**
- **Costo retrofit:** 1/17 zero-shot (SWIFT); 6/17 retrofit con fine-tuning (Mamba Drafters, STree, Mamba in the Llama, CALM, Medusa-1, EAGLE); 3/17 co-training obbligatorio (DeepSeek-V3 MTP, LayerSkip, Medusa-2); 4/17 con drafter esterno (DSpark, Ghidorah via Medusa head sul target, OpenInfer, Qualcomm); 1/17 infrastruttura speciale (QuantSpec gerarchica); 2/17 drafter n-gram/free (Snakes & Ladders).

Conclusione fattuale. L'argomento teorico bandwidth-amortization è supportato da ≥ 5 fonti indipendenti con formule quantitative coerenti. Il ceiling numerico $\approx 2-4\times$ del brief è allineato all'evidenza pubblicata ($2-3.3\times$ algoritmico puro). L'assenza di misure dirette su CPU consumer x86 + SSM target va dichiarata come **gap di evidenza Cat-A**; l'estrapolazione è giustificata dalla convergenza teorica ma non da replica sperimentale diretta. **Zona bianca specifica:** spec-decode con MLP ternario gated-dReLU come target — nessun paper trovato.

7. Bibliografia

Bibliografia numerata. Formato: [N] Autori (Anno). Titolo. Venue/arXiv-id. URL. Le citazioni inline nel testo si riferiscono a questi numeri.

- [1] DeepSeek-AI (Liu, A. et al.) (2024). *DeepSeek-V3 Technical Report*. arXiv:2412.19437. URL: <https://arxiv.org/abs/2412.19437> — codice: <https://github.com/deepseek-ai/DeepSeek-V3>.
- [2] Cheng, X.; Yu, X.; Shao, C.; Li, J.; Xiong, Y.; et al. (Peking University; DeepSeek-AI) (2026). *DSpark: Confidence-Scheduled Speculative Decoding with Semi-Autoregressive Generation*. URL: <https://www.alphaxiv.org/abs/2026.dspark> — codice: <https://github.com/deepseek-ai/DeepSpec> (MIT license).
- [3] Choi, D.; Oh, S.; Dingliwal, S.; Tack, J.; Kim, K.; Song, W.; Kim, S.; Han, I.; Shin, J.; Galstyan, A.; Katiyar, S.; Bodapati, S.B. (KAIST; Amazon AGI; Seoul National University) (2025). *Mamba Drafters for Speculative Decoding*. arXiv:2506.01206v1; EMNLP 2025 Findings. URL: <https://arxiv.org/abs/2506.01206>.

- [4] Wu, Y.; Dukler, Y.; Trager, M.; Achille, A.; Xia, W.; Soatto, S. (Amazon AWS) (2024). *Snakes and Ladders: Accelerating SSM Inference with Speculative Decoding*. PMLR vol. 262, pp. 292-304; NeurIPS 4th Efficient NLP and Speech Processing Workshop, 14 dicembre 2024. URL: <https://proceedings.mlr.press/v262/wu24a.html>.
- [5] Wu, Y.; Qin, Z.; Wong, A.; Soatto, S. (UCLA; Yale University) (2025). *STree: Speculative Tree Decoding for Hybrid State-Space Models*. arXiv:2505.14969v2; NeurIPS 2025 poster. URL: <https://arxiv.org/abs/2505.14969> — codice: <https://github.com/wyc1997/stree>.
- [6] Wang, J.; Paliotta, D.; May, A.; Rush, A.M.; Dao, T. (Cornell; University of Geneva; Together AI; Princeton) (2024). *The Mamba in the Llama: Distilling and Accelerating Hybrid Models*. arXiv:2408.15237; NeurIPS 2024. URL: <https://arxiv.org/abs/2408.15237>.
- [7] Wei, J.; Huang, Y.; Zhou, Y.; Jiang, J.; Du, J. (2025). *Ghidorah: Fast LLM Inference on Edge with Speculative Decoding and Hetero-Core Parallelism*. arXiv:2505.23219v1, 29 maggio 2025. URL: <https://arxiv.org/abs/2505.23219>.
- [8] OpenInfer (2025). *Boosting Local Inference with Speculative Decoding*. Engineering Blog, 5 agosto 2025. URL: <https://openinfer.io/news/2025-08-05-boosting-local-inference-with-speculative-decoding>. Fonte vendor blog, non peer-reviewed.
- [9] Zhang, Y.; Zhao, W.; Han, X.; Zhao, T.; Xu, W.; Cao, H.; Zhu, C. (2025). *Speculative Decoding Meets Quantization: Compatibility Evaluation and Hierarchical Framework Design*. arXiv:2505.22179v1, 28 maggio 2025. URL: <https://arxiv.org/abs/2505.22179> — codice: <https://github.com/AI9Stars/SpecMQuant>.
- [10] Vaidhyanathan, N. (Qualcomm) (2024). *How to Quadruple LLM Decoding Performance with Speculative Decoding (SpD) and Microscaling (MX) Formats on Qualcomm Cloud AI 100*. Qualcomm Developer Blog, marzo 2024. URL: <https://www.qualcomm.com/developer/blog/2024/03/quadruple-llm-decoding-performance-speculative-decoding-spd-microscaling-mx-formats>. Fonte vendor blog, non peer-reviewed.
- [11] Elhoushi, M.; Shao, D.; Singh, S.P.; Sevgen, R.M.; Dong, N.; McCallum, D.G.; Shrivastava, A.; Yu, L.; Zettlemoyer, L.; Levy, O.; Goyal, N.; Wehrstedt, L.; Joulin, A.; Çelikyilmaz, A.; Grave, E.; Choudhury, A.; Verma, N.; Shwartz-Ziv, R.; Sun, Z.; El-Houshi, Y.; Fernandez, J.; Lin, T.-Y.; Chandra, J.; Rabbat, M.; Chandra, V. (Meta AI) (2024). *LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding*. ACL 2024 (Long). arXiv:2404.16710. URL: <https://arxiv.org/abs/2404.16710> — codice: <https://github.com/facebookresearch/LayerSkip>.
- [12] Schuster, T.; Fisch, A.; Gupta, J.; Dehghani, M.; Dua, D.; Hou, H.; Zettlemoyer, L.; Joshi, M.; Peters, M.E.; Cohen, W.W.; Borgeaud, S. (Google) (2022). *Confident Adaptive Language Modeling*. NeurIPS 2022 (Oral). arXiv:2207.07061. URL: <https://arxiv.org/abs/2207.07061>.
- [13] Xia, H.; Yang, Z.; Qi, S.; Liu, Y.; Ge, T.; Wang, P.; Lin, Y.; Sui, Z. (Huawei; Peking University) (2025). *SWIFT: On-the-Fly Self-Speculative Decoding for LLM Inference Acceleration*. ICLR 2025. arXiv:2410.06916. URL: <https://arxiv.org/abs/2410.06916>.
- [14] Cai, T.; Li, Y.; Geng, Z.; Peng, H.; Lee, J.D.; Chen, D.; Dao, T. (Princeton; UIUC) (2024). *Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads*. ICML 2024. arXiv:2401.10774. URL: <https://arxiv.org/abs/2401.10774> — codice: <https://github.com/FasterDecoding/Medusa>.

- [15] Li, Y.; Wei, F.; Zhang, C.; Zhang, H. (2024). *EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty*. arXiv:2401.15077. URL: <https://arxiv.org/abs/2401.15077> — codice: <https://github.com/SafeAILab/EAGLE>.
- [16] Li, Y.; Wei, F.; Zhao, J.; Zhang, C.; Zhang, H. (2024). *EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees*. arXiv:2406.16858. URL: <https://arxiv.org/abs/2406.16858>.
- [17] Tiwari, R.; Xi, H.; Tomar, A.; Hooper, C.; Kim, S.; Horton, M.; Najibi, M.; Mahoney, M.W.; Keutzer, K.; Gholami, A. (UC Berkeley; Apple) (2025). *QuantSpec: Self-Speculative Decoding with Hierarchical Quantized KV Cache*. ICML 2025. URL: <https://machinelearning.apple.com/research/quantspec>.

Riferimenti secondari (citati nel testo, non estrazioni primarie)

- [18] DeepSeek-AI (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. arXiv:2501.12948, 22 gennaio 2025. URL: <https://arxiv.org/abs/2501.12948>. **Escluso** dal dossier: paper puramente RL, nessun claim spec-decode.
- [19] SpecMamba (2025). *SpecMamba: Speculative Decoding for Mamba on FPGA*. arXiv:2509.19873. Citato in sintesi per framing memory-bound SSM-specifico.
- [20] Spector, B.; Re, C. (2023). *Staged Speculative Decoding for Small-Batch On-Device LLM Inference*. arXiv:2308.04623. Citato in sintesi come fonte fondativa del framing *small-batch* = *memory-bound* = *spec-decode* conveniente. 3.16× su GPT-2-L 762M.
- [21] Xu et al. (2024). *EdgeLLM: Accelerating LLM Inference on Mobile Devices*. IEEE TMC 2024, DOI: 10.1109/TMC.2024.3513457. Citato indirettamente via SLED e Lever; PDF non estraibile in questa sessione. Da recuperare per chiusura del gap *flash-backed mobile LLM*.
- [22] Yang, N.; Lee, D.; Majumder, R.; Wei, F. (Microsoft) (2023). *In-Context Learning-Assisted Language Model Decoding (LLMA)*. arXiv:2304.04487. Citato per completezza — **non** self-speculative nel senso stretto (richiede retrieved document come draft).
- [23] Svirschevski, R. et al. (Yandex) (2024). *SpecExec: Massively Parallel Speculative Decoding for Mixture-of-Experts Hypothesis Evaluation*. NeurIPS 2024; arXiv:2406.02532. Citato per completezza — focus su consumer GPU + RAM offload (50B+), non self-speculative, non CPU.